SINGLE-USE FUNCTION SPACES FORM A MONOIDAL CLOSED CATEGORY AFTER THE QUOTIENT CONSTRUCTION

MIKOŁAJ BOJAŃCZYK, LÊ THÀNH DŨNG (TITO) NGUYỄN, RAFAŁ STEFAŃSKI

Editorial scope. Only named new single-use result counted. Authors identify vector-space section as old technical material; it and all remaining paper content are retained uncounted context.

Allowed standard prerequisites (editorial).

- Nominal sets, oligomorphic relational atoms, linear types, monoidal categories, game semantics and finite first-order theories. Full original definitions and original bbl retained. Prior orbit-finitely-spanned vector-space material in vector.tex and vector-appendix.tex is retained uncounted context because authors explicitly identify it as old BKM21 technical material.

1. Introduction

The class of orbit-finite sets is a class of sets that contains all finite sets and some infinite sets, but still shares some properties with the class of finite sets. The idea, which dates back to Fraenkel–Mostowski models of set theory, is to begin with an infinite set $\mathbb{A}$ of *atoms* or *urelements*. We think of the atoms as being names, such as Eve or John, and atoms can only be compared with respect to equality. Intuitively speaking, an orbit-finite set is a set that can be constructed using the atoms, such as $\mathbb{A}^2$ or $\mathbb{A}^*$, subject to the constraint that there are finitely many elements up to renaming atoms. For example, $\mathbb{A}^2$ is orbit-finite because it has two elements up to renaming atoms, namely (John, John) and (John, Eve), while $\mathbb{A}^*$ is not orbit-finite, because the length of a sequence is invariant under renaming atoms, and there are infinitely many possible lengths. For a survey on orbit-finite sets, see [3].

The notion of orbit-finiteness can be seen as an attempt to find an appropriate notion of finiteness for the nominal sets of Gabbay and Pitts [21]. This attempt emerged from the study of computational models such as monoids [4] and automata [5] over infinite alphabets. Since automata are also the main use case for the present paper, we illustrate orbit-finiteness using an automaton example.

Example 1.1 (An orbit-finite automaton)**.** Let $L \subseteq \mathbb{A}^*$ be the language of words in which the letter from the first position does not appear again. This language contains John $\cdot$ Mark $\cdot$ Mark $\cdot$ Eve, because John does not reappear, but it does not contain John $\cdot$ Mark $\cdot$ John. To recognize this language, we can use a deterministic automaton, which uses its state to remember the first letter. In this automaton, the input alphabet is $\Sigma = \mathbb{A}$ and the state space is $Q = 1 + 1 + \mathbb{A}$. In this state space, there are two special states, namely the initial state and a rejecting error state, and furthermore there is one state for each atom $a \in \mathbb{A}$, which represents a situation where the first letter was a but it has not been seen again yet. This state space is infinite; but it is orbit-finite, since each of the three components in Q represents a single orbit, with the orbits of the 1 components being singletons.◀

Date: arxiv:2404.05265v1; CC BY 4.0.

Orbit-finite sets have many advantages, which ensure that they are a good setting for automata theory, and discrete mathematics in general. For example, an orbit-finite set can be represented in a finite way [3], which ensures that it becomes meaningful to talk about algorithms that input orbit-finite sets, such as an emptiness check for an automaton. Also, orbit-finite sets are closed under taking disjoint unions and products, which ensures that natural automata constructions, such as the union of two nondeterministic automata or the product of two deterministic automata can be performed.

However, orbit-finite sets do not have all the closure properties of finite sets. Notably missing is the powerset construction, and more generally taking function spaces. For example, if we look at the powerset of $\mathbb{A}$, then this powerset will not be orbit-finite, since already the finite subsets give infinitely many orbits (two finite subsets of different sizes will be in different orbits). The lack of powersets means that one cannot do the subset construction from automata theory, and in particular deterministic and nondeterministic automata are not equivalent. This non-equivalence was known from the early days of automata for infinite alphabets [16], and in fact, some decision problems, such as equivalence, are decidable for deterministic automata but undecidable for nondeterministic automata [17]. Another construction that fails is converting a deterministic automaton into a monoid [4, p. 221]; this is because function spaces on orbit-finite sets are no longer orbit-finite, as explained in the following example.

Example 1.2 (Failure of the monoid construction)**.** Let us show that the automaton from Example 1.1 cannot be converted into a monoid. The standard construction would be to define the monoid as the subset $M \subseteq Q \to Q$ of all state transformations, namely the subset generated by individual input letters. Unfortunately, this construction does not work. This is because in order for two input words to give the same state transformation, they need to have the same set of letters that appear in them. In particular, the corresponding set of set transformations is not orbit-finite, for the same reason as why the finite powerset is not orbit-finite. Not only does the standard construction not work, but also this language is not recognized by any orbit-finite monoid.◀

An attempt to address this problem was provided in [25, 26, 7], by using *single-use* functions. The idea, which originates in linear types and linear logic, is to restrict the functions so that they use each argument at most once. For example, consider the following two functions that input atoms and output Booleans:

$$a \in \mathbb{A} \mapsto \begin{cases} \text{true} & \text{if } a = \text{John} \\ \text{false} & \text{otherwise} \end{cases} \qquad a \in \mathbb{A} \mapsto \begin{cases} \text{true} & \text{if } a = \text{John or } a = \text{Eve} \\ \text{false} & \text{otherwise} \end{cases}$$

The first function is single-use, since it compares the input atom to John only, while the second function is not single-use, since it requires two comparisons, with John and Eve. Here is another example, which shows that the problems with the monoid construction from Example 1.2 could be blamed on a violation of the single-use condition.

Example 1.3. Consider the transition function of the automaton in Example 1.1, which inputs a state in $1 + 1 + \mathbb{A}$ together with an input letter from $\mathbb{A}$, and returns a new state. This function is not single use. Indeed, if the state is in $\mathbb{A}$, then the transition function compares it for equality with the input letter; but if the comparison returns true, another copy of the old state must be kept as the new state for future comparisons. ◀

If one restricts attention to functions that are single-use, much of the usual robustness of automata theory is recovered, with deterministic automata being equivalent to monoids, and both being equivalent to two-way deterministic automata [7].

Despite the success of the single-use restriction in solving automata problems, one would ideally prefer a more principled approach, in which instead of defining single-use automata, we would define a more general object, namely single-use sets and functions. Then the definitions of automata and monoids, as well theorems speaking about them, should arise automatically as a result of suitable closure properties of the sets and functions.

This approach was pursued in [26], in which a *category* of orbit-finite sets with single-use functions was proposed. In this corresponding category, one can represent the set of all single-use functions between two orbit-finite sets X and Y as a new set, call it $X \Rightarrow Y$, which is also orbit-finite. However, as we will see later in this paper, this proposal is not entirely satisfactory, since it fails to account for standard operations that one would like to perform on function spaces, most importantly partial application (currying). Note that partial application is crucial for converting an automaton into a monoid, since the monoid consists of partially applied transition functions, in which the input word is known, but the input state is not. In the language of category theory, the proposal from [26] failed to be a monoidal closed category.

Let us mention there have been several works using category theory to generalize classical operations on automata, such as the coalgebraic "generalized powerset construction" [24]. The closest to the philosophy we exposed above might be the setting introduced by Colcombet and Petrişan [9], where automata in different categories are compared (see [10, 2] for applications). Within this setting, Nguyễn and Pradic have studied some properties of automata over monoidal closed categories [18, Sections 1.2.3 and 4.7–4.8] as part of their research on "implicit automata"[1] [20, 19, 18, 22].

Contributions of this paper. The main contribution of this paper is to propose a notion of single-use sets and functions, which extends the proposal from [26], but which is rich enough to be closed under taking function spaces. More formally, we propose a category of single-use functions between orbit-finite sets equipped with additional metadata, and we prove that this category is symmetric monoidal closed.

The main idea is to follow the tradition of linear types, and to distinguish two kinds of products, namely[2] $X \otimes Y$ and $X \& Y$. Thanks to this distinction, the function space can be built so that the appropriate operations on functions, namely application and currying, can be implemented in a single-use way.

Our proposed category is strongly inspired by linear types, and the proof that it is symmetric monoidal closed uses a form of *game semantics*[3] – a tool that we take

[1]This line of work is about relating the expressive power of automata and typed λ-calculi. In [19, 18], in addition to results on arbitrary symmetric monoidal closed categories, such a category of single-use assignments on string-valued registers is built and used to relate a register-based string transducer model to a λ-calculus with linear types. Indeed, symmetric monoidal closed categories are famous for providing denotational semantics for the linear λ-calculus. Similarly, our results here could serve to characterize the languages of words with atoms from [7] via some typed λ-calculus.

[2]Categorically, & is the cartesian product while $\otimes$ is a monoidal product.

[3]In this paper, the game semantics will only appear in the appendix, as they are part of our proofs rather than our main claims. That said, let us point to the lecture notes [1, 15] as references for the category of "simple games" upon which we build. For a recent survey of modern game semantics, see [8].

In the work mentioned in Section 1, the fact that "register contents do not influence the control flow" leads to a much simpler construction, similar to Dialectica categories [11] (which can be seen as a form of game semantics with very short plays), where no quotient is needed. Our Theorem 3.8

from programming language theory. However, to the best of our knowledge, it is an original idea to have an infinite but orbit-finite base type $\mathbb{A}$, and to observe that all constructions in game semantics are consistent with orbit-finiteness. We believe that the resulting category deserves further study, and that it is an interesting and non-trivial example of a category representing "finite" objects.

Along the way, we provide examples of how the category can be useful in automata theory. Our main example is converting a deterministic automaton into a monoid, but another example is modelling two-way deterministic automata, and deciding their emptiness.

An important property of our construction is that it is generic. In fact, instead of having a set that is equipped with equality only, one could apply the construction to any relational structure, e.g. real arithmetic $(\mathbb{R}, +, \times, <)$, as long as the structure is given using relations and not functions. Under further assumptions on the structure, such as having a decidable first-order theory (which holds for real arithmetic) and being oligomorphic (which does not hold for real arithmetic but does hold for the rational numbers with their linear order), further benefits in the resulting single-use category can be derived, such as having an emptiness algorithm for deterministic two-way automata.

Finally, as a minor contribution, we present an alternative solution for the problem of function spaces, which is to use vector spaces of orbit-finite dimension. This is a minor contribution as far as the present paper is concerned, because the technical tools were developed already in [6], and the only contribution – if any – of this paper is one of perspective, namely framing it as a symmetric monoidal closed category. An advantage of the vector space category is its simplicity, and the fact that it is "bigger" in the following sense. The two solutions for function spaces discussed in this paper, namely the single-use solution and the vector space solution, sit on both sides of the classical category of orbit-finite sets, as witnessed by two faithful functors, one from the single-use category to the orbit-finite category, and one from the orbit-finite category to the vector space category. The generality of vector spaces comes at a price, though. As mentioned before, the single-use construction can be applied to any structure, and the orbit-finite benefits can be derived for all ω-categorical structures (this is the standard assumption in the study of orbit-finiteness). In contrast, the vector space construction is more brittle, and it fails for certain ω-categorical structures such as the Rado graph. Another disadvantage of the vector space category is that it is not traced with respect to the coproduct, unlike the single-use category, which precludes the applications to two-way automata.

2. Sets with atoms

We begin with a brief introduction to orbit-finite sets. For a more detailed treatment, see [3].

Fix for the rest of this paper a countably infinite set $\mathbb{A}$, whose elements will be called atoms. We assume that this set has no other structure except for equality, which will mean that we will only be interested in notions which are *equivariant*, i.e. invariant under renaming atoms. For example, $\mathbb{A}$ has only two equivariant subsets, namely the empty and full subsets. On the other hand, the set $\mathbb{A}^2$ has four equivariant subsets; this is because any subset containing (Eve, Eve) must contain all pairs on the diagonal, and any subset containing (Eve, John) must contain all

might also be provable by representing single-use functions as λ-terms (or programs in some richly structured syntactic formalism) instead of strategies over games. Indeed, it is a classical fact that a simple type is inhabited by finitely many linear λ-terms up to β-conversion, and variations on this fact have been used in the literature to relate automata and λ-calculus [20, 12].

pairs outside the diagonal. In order to meaningfully speak about equivariant subsets, we must be able to have an action of atom renamings on the set, as formalized in the following definition. The finite support condition is a technical condition that ensures that the action is well-behaved; this condition dates back to the work of Fraenkel and Mostowski, and is explained in the survey texts [21, 3].

Definition 2.1 (Set with atoms)**.** A *set with atoms* is a set X, equipped with an action of the group of atom renamings, subject to the following *finite support condition*: for every $x \in X$ there is a finite set of atoms, such that if an atom renaming π fixes all atoms in the set, then it also fixes x.

The idea is that a set with atoms is any kind of object that deals with atoms, such as the set $\mathbb{A}^*$ of all words over the alphabet $\mathbb{A}$, or the family of finite subsets of $\mathbb{A}$. Among such objects, we will be interested in those which are "finite". This will be formalized by saying that there are finitely many orbits, as described below. Define the *orbit* of an element x in a set with atoms to be the elements that can be obtained from x by applying some atom renaming. For example, in the set $\mathbb{A}^2$, the orbit of (John, Eve) contains (Mark, John), but it does not contain (John, John). The orbits form a partition of a set with atoms.

Definition 2.2 (Orbit-finite set)**.** A set with atoms is called *orbit-finite* if it has finitely many orbits.

A typical example of an orbit-finite set is $\mathbb{A}^4$, or more generally any polynomial expression such as $\mathbb{A}^4 + \mathbb{A}^3 + \mathbb{A}^3 + 1$. Here, 1 represents the set of zero-length sequences; this set has a unique element which is its own orbit. For example, $\mathbb{A}^3$ has five orbits, because there are five possible ways of choosing a pattern of equalities in a sequence of three names. On the other hand, $\mathbb{A}^*$ has infinitely many orbits, since sequences of different lengths are necessarily in different orbits. The family of finite subsets of $\mathbb{A}$ is also not orbit-finite, because subsets of different sizes are in different orbits. The full powerset $\mathsf{P}\mathbb{A}$ is not even a legitimate object in our setting, because some of its elements, i.e. some subsets of $\mathbb{A}$, violate the finite support condition.

2.1. **Finiteness of function spaces.** As mentioned above, orbit-finite sets can be seen as a certain generalization of finite sets. They allow some, but not all, operations that can usually be done on finite sets. For example, orbit-finite sets are closed under disjoint unions $X + Y$ and products $X \times Y$. Another good property is that an orbit-finite set has only finitely many equivariant subsets (an equivariant subset is one that is invariant under the action of atom permutations). This is because an equivariant subset is a union of some of the finitely many orbits. This accounts for some of the good computational properties of orbit-finite sets. For example, nonemptiness is decidable for orbit-finite automata (see below), because the state space can be searched orbit by orbit. However, orbit-finite sets do not have orbit-finite function spaces, as explained in the following example.

Example 2.3. Consider the space of functions of type $\mathbb{A} \to \mathbb{A}$. What are the legitimate functions? One choice is that we only allow the equivariant functions. Under this choice, there is only one possible function, namely the identity function. However, as we will explain below, the more appropriate choice is the class of finitely supported functions, i.e. those that are invariant under all atom renamings that fix some finite set of atoms that depends only on the function. For example, the function

$$f(a) = \begin{cases} \text{Mark} & \text{if } a \in \{\text{John, Eve, Bill}\} \\ a & \text{otherwise} \end{cases}$$

is finitely supported, because it is invariant under all atom renamings that fix Mark, John, Eve and Bill. The space of finitely supported functions will be called the *finitely supported function space*. This space is equipped with an action of atom renamings. For example, if π is the atom renaming that swaps Mark with Adam, then applying it to the function f defined above gives the function $\pi(f)$ that has the same definition (or source code, if a programming intuition is to be followed), except that Mark is used instead of Adam.

In the case of $\mathbb{A} \to \mathbb{A}$, the finitely supported function space is not orbit-finite. Indeed, the condition $a \in \{\text{John, Eve, Bill}\}$ can be replaced by $a \in X$ for any finite set $X \subset \mathbb{A}$ of exceptional values, and two choices of X of different cardinalities will give us two functions in different orbits. ◀

In the above example, we explained how the finitely supported function space is not always orbit-finite. On the other hand, the equivariant function space will always be literally finite, if the input and output types are orbit-finite, because it will be an equivariant subset of the product $X \times Y$. So why do we insist on the finitely supported function space? A more principled reason will appear later in the paper, where we discuss symmetric monoidal closed categories, and where the finitely supported function space will turn out to be the right one. However, we can already give a simple reason, which appeals to automata theory, namely converting an automaton into a monoid. As we have seen in Example 1.2, when converting an automaton to a monoid, we will want to use partially applied transition functions, and such functions will be finitely supported but not equivariant.

The lack of function spaces is the problem addressed in this paper. We present two solutions of this problem. In each of the two solutions, we modify the notion of orbit-finite sets, by either restricting it or generalizing it, in a way that recovers function spaces.

3. Single-use sets and functions

We now turn to our first solution for the problem of orbit-finite function spaces. Our solution builds on the idea from [26, Section 2.2], which is to consider only functions that are single-use. We describe this idea in Section 3.1, and we show how it almost, but not quite, achieves function spaces. Then, in the rest of this section, we show how function spaces can be recovered by using a more refined type system.

3.1. Single-use functions over polynomial orbit-finite sets.

In the introduction, we have already given an intuitive description behind the single-use functions; these are functions that destroy the argument after any use of it, such as comparing it to a constant. We now give a more formal definition.

We do not define the single-use functions on all orbit-finite sets, but only a syntactically defined fragment, namely the *polynomial orbit-finite sets*, which are sets that can be generated from 1 and $\mathbb{A}$ using products $\times$ and disjoint unions $+$. Therefore, we will allow orbit-finite sets like $1+\mathbb{A}^2$, but we will not allow orbit-finite sets like the set of non-repeating pairs $\{(a,b) \mid a \neq b \in \mathbb{A}\}$ or the set of unordered pairs $\{\{a,b\} \mid a \neq b \in \mathbb{A}\}$. It is an open problem to find a satisfactory definition of single-use functions on all orbit-finite sets. (A simple hack is to use a quotienting construction, similar to Section B, but what we would really like to do is to identify some extra structure in a set, possibly an action of some yet unknown group or semigroup, which enables us to speak about single-use functions.)

Consider two polynomial orbit-finite sets X and Y. To define which functions $X \to Y$ are single-use, we use an inductive definition. We begin with certain functions that are considered single-use, such as the equality test of type $\mathbb{A} \times \mathbb{A} \to$

$1 + 1$. These functions are called the *prime functions*, and their full list is given in Figure 1. Next, we combine the prime functions into new ones using three combinators. The first, and most important, combinator is function composition. Then, we have two combinators for the two type constructors: if two functions $f_1 : X_1 \to Y_1$ and $f_2 : X_2 \to Y_2$ are single-use, then the same is true for:

$$f_1 \times f_2 : X_1 \times X_2 \to Y_1 \times Y_2 \qquad f_1 + f_2 : X_1 + X_2 \to Y_1 + Y_2$$
$$(x_1,x_2)\mapsto(f_1(x_1),f_2(x_2)) \qquad \mathrm{left}(x_1)\mapsto\mathrm{left}(f_1(x_1)) \quad \mathrm{right}(x_2)\mapsto\mathrm{right}(f_2(x_2)).$$

Crucially, the list of prime single-use functions does not contain the copying function $a \in \mathbb{A} \mapsto (a, a) \in \mathbb{A}^2$. Therefore, an alternative name for the single-use functions is *copyless*. If we added copying, then we would get all finitely supported functions [26, Lemma 23].

Function	**Type**	**Definition**
Functions about $\mathbb{A}$		
equality test	$\mathbb{A} \times \mathbb{A} \to 1 + 1$	$a, b \mapsto$ if $a = b$ then true else false
constant a	$1 \to \mathbb{A}$	$x \mapsto a$
identity	$\mathbb{A} \to \mathbb{A}$	$x \mapsto x$
Functions about $\times$		
commutativity of $\times$	$X \times Y \to Y \times X$	$x \times y \mapsto y \times x$
first projection	$X \times Y \to X$	$x \times y \mapsto x$
second projection	$X \times Y \to Y$	$x \times y \mapsto y$
append 1	$X \to X \times 1$	$x \mapsto x \times ()$
associativity of $\times$	$(X \times Y) \times Z \to X \times (Y \times Z)$	$(x \times y) \times z \mapsto x \times (y \times z)$
Functions about $+$		
first co-projection	$X \to X + Y$	$x \mapsto \mathrm{left}(x)$
second co-projection	$Y \to X + Y$	$y \mapsto \mathrm{right}(y)$
co-diagonal	$X + X \to X$	$\begin{cases} \mathrm{left}(x) \mapsto x \\ \mathrm{right}(x) \mapsto x \end{cases}$
commutativity of $+$	$X + Y \to Y + X$	$\begin{cases} \mathrm{left}(x) \mapsto \mathrm{right}(x) \\ \mathrm{right}(y) \mapsto \mathrm{left}(y) \end{cases}$
associativity of $+$	$(X + Y) + Z \to X + (Y + Z)$	$\begin{cases} \mathrm{left}(\mathrm{left}(x)) \mapsto \mathrm{left}(x) \\ \mathrm{left}(\mathrm{right}(y)) \mapsto \mathrm{right}(\mathrm{left}(y)) \\ \mathrm{right}(z) \mapsto \mathrm{right}(\mathrm{right}(z)) \end{cases}$
Distributivity		
$+$ distributes over $\times$	$X \times (Y + Z) \to (X \times Y) + (X \times Z)$	$\begin{cases} x \times (\mathrm{left}(y)) \mapsto \mathrm{left}(x \times y) \\ x \times (\mathrm{right}(z)) \mapsto \mathrm{right}(x \times z) \end{cases}$

TABLE 1. The prime single-use functions for polynomial orbit-finite sets X, Y and Z.

Example 3.1. Consider function of type $\mathbb{A}^3 \to \mathbb{A}$ which inputs a triple (a, b, c) of atoms and returns a if c is equal to Mark, and b otherwise. This function is a single-use function. It is obtained by composing the six functions listed below:

Function	Type after function
Append 1.	$\mathbb{A} \times \mathbb{A} \times \mathbb{A} \times 1$
Replace added 1 with Mark using the constant function.	$\mathbb{A} \times \mathbb{A} \times \mathbb{A} \times \mathbb{A}$
Apply the equality test to the last two components.	$\mathbb{A} \times \mathbb{A} \times (1 + 1)$
Distribute.	$\mathbb{A} \times \mathbb{A} \times 1 + \mathbb{A} \times \mathbb{A} \times 1$
Project to first and second components, respectively.	$\mathbb{A} + \mathbb{A}$
Co-diagonal	$\mathbb{A}$

To justify this description, one should also show that the six functions are single-use. Three of the functions, namely append 1, distributivity and co-diagonal are prime functions. The other three are obtained by combining prime functions using the combinators. For example, the equality test is paired, using the combinator for $\times$, with the identity on the remaining two atoms. ◀

The design goal of the single-use restriction is to have orbit-finite function spaces. The rough idea is that a single-use function can only use a bounded number of atoms in its source code, which guarantees orbit-finiteness of the function space.

Example 3.2. Consider function of type $\mathbb{A} \to 1 + 1$, which can be seen as subsets of the atoms, with $1 + 1$ representing the Booleans. We will consider two function spaces: the larger space of all finitely supported functions, and the smaller space of single-use functions.

A function in the larger space is any finitely supported subset of the atoms; such subsets are the same as the finite and co-finite subsets. Therefore, the larger function space admits an equivariant bijection with a disjoint union of two copies of the finite powerset $\mathsf{P}_{\text{fin}}\mathbb{A}$, in particular it is not orbit-finite.

Consider now the smaller single-use function space. There are four possible functions of this kind: (a) always return true; (b) always return false; (c) check for equality with some fixed atom a; (d) check for disequality ($\neq$) with some fixed atom a. Therefore, the set of single-use functions admits an equivariant bijection with the orbit-finite set $1 + 1 + \mathbb{A} + \mathbb{A}$. ◀

The above example shows that the space of single-use functions of some type $X \to Y$ is orbit-finite, and in fact it can be described using a polynomial orbit-finite set. This is true for every choice of polynomial orbit-finite sets X and Y, as proved in [26, Theorem 5], and illustrated in the following example.

Example 3.3. Assume that the input type X is some power of the atoms $\mathbb{A}^k$, and the output type Y does not use atoms, e.g. it is $Y = 1 + 1$. The assumption on the input type can be made without loss of generality using distributivity, while the assumption on the output type is a proper restriction, but it will allow us to skip some technical details of the general construction while retaining the important intuitions. We describe below a type that represents all single-use functions from $\mathbb{A}^k$ to Y; we shall denote it by $\mathbb{A}^k \Rightarrow Y$. Note that $\Rightarrow$ is *not* a primitive type constructor in our grammar of types; it is a notation that stands for the inductive construction below.

This type is defined by induction on k. In the base case of $k = 0$ we simply need to give a value from the output type, and therefore $\mathbb{A}^0 \Rightarrow Y$ is the same as Y. Consider now the induction step of $k > 0$. We observe that a single-use function that inputs $\mathbb{A}^k$ must begin with some equality test, and then continue with one of two single-use functions that have fewer arguments (one for the case when the equality test returns true, and one for the other case). This observation leads to

the following definition of the type $\mathbb{A}^k \Rightarrow Y$:

$$\underbrace{\coprod_{i\in\{1,\dots,k\}} \mathbb{A} \times (\mathbb{A}^{k-1} \Rightarrow Y) \times (\mathbb{A}^{k-1} \Rightarrow Y)}_{\substack{\text{starts by comparing } i\text{-th} \\ \text{coordinate to some constant}}} \quad + \quad \underbrace{\coprod_{i,j\in\{1,\dots,k\}} (\mathbb{A}^{k-2} \to Y) \times (\mathbb{A}^{k-2} \to Y)}_{\substack{\text{starts by comparing the} \\ i\text{-th and } j\text{-th coordinates}}}.$$

Note that the above representation of the function space is not necessarily unique, i.e. the same function can be represented in several different ways. For example, the order in which equality tests are performed will matter for the representation, but might not matter for the function. This is not something that we worry about, and we will use function spaces with non-unique representations in the paper, see also Section B for how we deal with non-uniqueness. ◀

Problem with currying. Unfortunately, the proposal illustrated in Example 3.3 and described in more detail in [26] does not give a satisfactory solution to the problem of function spaces. The problem is that the set of representations $X \Rightarrow Y$ should also support operations on functions. More specifically, we should be able to indicate single-use operations which do the following:

evaluation:: a single-use function from $(X \Rightarrow Y) \times X$ to Y which inputs a representation of a function and applies it to an argument;

composition:: a function from $(X \Rightarrow Y) \times (Y \Rightarrow Z)$ to $(X \Rightarrow Z)$ which inputs the representations of two functions and returns a representation of their composition.

currying:: for each single-use function from $X \times Y$ to Z, there should be a single-use function from X to $Y \Rightarrow Z$ which inputs a first argument and returns a representation of the partially applied function;

Only in the presence of all of these operations can we speak of a function space, and the corresponding category can be called closed. (Composition can be obtained through evaluation and currying, so the essential operations are evaluation and currying.) The following example shows that the currying operation is not single-use, and therefore the space of single-use functions as defined in this section is not closed.

Example 3.4. Consider the single-use function

$$f : \mathbb{A} \times \mathbb{A} \to 1 + 1 \qquad (a, b) \mapsto \begin{cases} \text{result of test } a = \text{Mark} & \text{if } b = \text{Eve} \\ \text{result of test } a = \text{John} & \text{otherwise.} \end{cases}$$

The currying of this function, is a new function which maps a first argument $a \in \mathbb{A}$ to the partially applied function $f(a, _)$. This currying is

$$a \mapsto \begin{cases} b \mapsto b = \text{Eve} & \text{if } a = \text{Mark} \\ b \mapsto b \neq \text{Eve} & \text{if } a = \text{John} \\ b \mapsto \text{false} & \text{otherwise} \end{cases}$$

Recall that in Example 3.2 we showed that the space of single-use functions of type $\mathbb{A} \to 1 + 1$ can be represented as $1 + 1 + \mathbb{A} + \mathbb{A}$. If we use this representation, then the currying of the function f is not single-use, because we need to compare the input atom a to two constants, Mark and John. If we use the representation from Example 3.3, then the corresponding type will be $\mathbb{A} \otimes (1 + 1)^2$, but the problems with currying will persist. ◀

For similar reasons, the function space, we proposed above, will also not support function composition, which means that it cannot be used to convert automata into

single-use monoids, as we would like to do, since the resulting monoid would need to use function composition as its monoid operation.[4]

To solve the problems above, we will introduce a more refined type system, which is based on linear types. The main idea is to pay more attention to type in Example 3.3. In that definition, we describe a single-use function by specifying the first equality test that it makes, and then giving two descriptions of the functions that will be used in each of the two possible outcomes of the equality test. The main observation is that these two outcomes are mutually exclusive, and therefore we intend to use only one of the two descriptions. For this reason, we will use a type constructor & that comes from linear logic. The intended meaning is that an object of type $X\&Y$ consists of two objects, but with the ability to use only one of them. Since linear logic uses $\otimes$ for the product that we have so far denoted by $\times$, we will also follow that convention. Using these two kinds of products, the appropriate type for Example 3.3 will now become:

$$\coprod_{i\in\{1,\ldots,k\}} \mathbb{A}\otimes((\mathbb{A}^{k-1}\Rightarrow Y)\&(\mathbb{A}^{k-1}\Rightarrow Y)) \quad + \quad \coprod_{i,j\in\{1,\ldots,k\}} (\mathbb{A}^{k-2}\to Y)\&(\mathbb{A}^{k-2}\to Y).$$

Under this definition, the problems from Example 3.4 will be solved, at least for the particular type considered in that example. However, by introducing a new type constructor, we will have to redefine the single-use functions, and then we will have to give a representation of functions that allow this new type constructor, without incurring the need to add any other new type constructors. This is the subject of the next section.

3.2. **Linear types and single-use functions on them.** As mentioned above, to solve the problems with single-use function spaces, we will consider a type system with two kinds of products, as in the following definition.

Definition 3.5 (Linear types)**.** A *linear type* is any expression constructed from the atomic types 1 and $\mathbb{A}$ using three[5] binary type constructors $+, \&$ and $\otimes$.

In our linear types, it is only the products that are differentiated, while $+$ comes in only one version. Here is the intuitive explanation of the difference between the two kinds of products, following Girard [13, p.2]. Having a pair $x\otimes y$ is like having the ability of using both components x and y. On the other hand, having a pair $x\&y$ is like having the ability to use one of the two components, at our choice, but not both at once. For example, the input type of the equality test will be $\mathbb{A}\otimes\mathbb{A}$ not $\mathbb{A}\&\mathbb{A}$, since the test will need to consume both arguments. This intuition can only go so far; for example, it is not entirely clear what "our choice" means. We revisit this intuition in the appendix, where game semantics will be used to indicate who makes which choices.

We think of each linear type X as representing a set $[\![X]\!]$, as defined below:

$$[\![1]\!]=1 \quad [\![\mathbb{A}]\!]=\mathbb{A} \quad [\![X+Y]\!]=[\![X]\!]+[\![Y]\!] \quad [\![X\otimes Y]\!]=[\![X\&Y]\!]=[\![X]\!]\times[\![Y]\!].$$

All sets that arise in this way will be polynomial orbit-finite sets. Note that the two kinds of product represent the same set, namely the set of pairs in the usual set-theoretic sense. However, the two type constructors will be different, because different functions will be allowed to operate on them. As the expression goes, "the

[4]This problem is solved in [7] and [26] in a different way, namely by showing that every orbit-finite monoid necessarily divides a single-use monoid, using a Krohn-Rhodes construction. However, this construction is difficult and delicate, in particular it does not work for atoms that have more structure than equality alone. In contrast, the proposal that we give in this paper works for other kinds of atoms, as discussed in Section 4.

[5]We set up our type system without using the multiplicative disjunction $\parr$ of linear logic – morally, we take our inspiration from intuitionistic linear logic, rather than classical linear logic.

proof of the pudding is in the eating"; in this case the pudding is the types and the eating is the functions.

As we did in Section 3.1, the single-use functions will be defined in terms of prime functions and combinators. The combinators are the same, except that instead of $f_1 \times f_2$ we now have two ways of pairing functions, using $\otimes$ and &. The prime functions are inherited from the previous system, with $\times$ understood as $\otimes$, together with a few new functions for &, as described in Table 2. This is summarized in the following definition.

Definition 3.6 (Single-use functions)**.** The class of single-use functions is the least class of functions with the following properties:

(1) It contains the functions from Tables 1 and 2, with $\times$ in Table 1 understood as $\otimes$;
(2) It is closed under composition, as well as under combining functions using $+$, $\otimes$ and &.

Function	**Type**	**Definition**
diagonal	$X \to X\&X$	$x \mapsto x\&x$
first projection	$X\&Y \to X$	$x\&y \mapsto x$
second projection	$X\&Y \to Y$	$x\&y \mapsto y$
& distributes over $\otimes$	$X \otimes (Y\&Z) \to (X \otimes Y)\&(X \otimes Z)$	$x \otimes (y\&z) \mapsto (x \otimes y)\&(x \otimes z)$
& distributes over $+$	$X + (Y\&Z) \to (X\&Y) + (X\&Z)$	$\begin{cases} x\&\mathrm{left}(y) \mapsto \mathrm{left}(x\&y) \\ x\&\mathrm{right}(z) \mapsto \mathrm{right}(x\&z) \end{cases}$

TABLE 2. Prime single-use functions that involve &.

Formally speaking, a single-use function consists of an input linear type X, an output linear type Y, and a function between the sets $[\![X]\!]$ and $[\![Y]\!]$ that is generated using the prime functions and combinators from the above definition. As was the case in Section 3.1, all single-use functions are finitely supported. Therefore, one can think of the single-use functions of type $X \to Y$ as being a subset of the set of all finitely supported functions from $[\![X]\!]$ to $[\![Y]\!]$. This subset is strict: as we will see, the space of single-use functions will be orbit-finite, unlike the space of all finitely supported functions. We will be thinking of the single-use functions as a category.

Definition 3.7 (Category of single-use sets)**.** The category of single-use sets is:

(1) The objects are linear types, as per Definition 3.5.
(2) Morphisms between types X and Y are single-use functions from $[\![X]\!]$ to $[\![Y]\!]$.

In the very definition of the above category, there is a faithful functor to the category of orbit-finite sets with finitely supported functions. This functor maps objects X to their underlying sets $[\![X]\!]$, which are orbit-finite sets, and it maps morphisms to the corresponding functions. The functions seen to be finitely supported, and the functor is faithful by definition, since the morphisms in Definition 3.7 are defined to be single-use functions.

The main technical result of this paper is that the category of single-use sets has function spaces, as stated in the following theorem. The appropriate product will be $\otimes$, and not &. Since the Cartesian product in our category is & and not $\otimes$, this means that the result we are targeting is symmetric monoidal closed

with respect to $\otimes$, and not Cartesian closed. (The same situation will happen for vector spaces later in this paper.) Our theorem stops a bit short of saying that the category is monoidal closed, since several different elements of the function space might represent the same function. This non-uniqueness of representation can be overcome by quotienting the function space by an equivalence relation that identifies two elements if they represent the same single-use function; this is explained in Appendix B. For now, we stick with non-uniqueness of representation, and we state the theorem as follows.

Theorem 3.8. *Let V and W be objects (i.e. linear types). There exists an object, denoted by $V \Rightarrow W$, and a morphism (i.e. a partial single-use function)*

$$eval : (V \Rightarrow W) \otimes V \to W$$

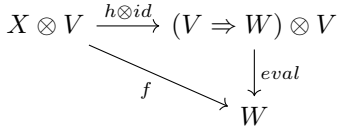
with the following property. For every morphism

$$f : X \otimes V \to W$$

there is a (not necessarily unique) morphism

$$h : X \to (V \Rightarrow W)$$

such that the following diagram commutes:



The above theorem is the main technical contribution of this paper, and its proof is presented in Appendix A. The difficulty in the proof is finding a representation of the single-use functions that is rich enough to capture all functions, but simple enough to be described by a linear type (in particular, the corresponding set will be orbit-finite). In Section 3.1, when the types did not have &, we could pull off a relatively simple construction, which was possible mainly due to the strong distributivity rules that allowed converting each type into a normal like $\mathbb{A}^{n_1} + \cdots + \mathbb{A}^{n_\ell}$. In the presence of &, the distributivity rules are not as strong, and the way in which a single-use program can interact with its input is rather subtle. Our solution, and the technical core of this paper, is to use game semantics, appropriately extended to describe the type $\mathbb{A}$ and the operations on it that are allowed. This solution is presented entirely in the appendix.

4. Beyond equality

So far, we have only studied the case when the atoms are equipped with equality only. Consider now a more general case: let $\mathbb{A}$ be any relational structure, i.e. any set equipped with relations (but not functions). For example, we could use the rational numbers with their linear order. Another example would be Presburger arithmetic, i.e. the integers $\mathbb{N}$ together with addition $+$. Since we want to have relations only, we think of addition as a ternary relation $x + y = z$.

The construction of the single-use category from Section 3 is generic enough so that it be generalized to any relational structure, and not just atoms with equality. (In fact, there is also a suitable generalization for function symbols, but we do not prove anything about it, so we do not talk about it.)

Definition 4.1 (Single-use functions over a relational structure)**.** Let $\mathbb{A}$ be a relational structure. The *single-use category over* $\mathbb{A}$ is defined in the same way as in Definition 3.7, except that the list of prime functions is extended with one prime

function $\mathbb{A}^n \to 1 + 1$ for every n-ary relation in the vocabulary of $\mathbb{A}$. (Here, the power $\mathbb{A}^n$ uses $\otimes$.)

Not only is the definition of the category generic, but the same is true for the proof that function spaces exist. The generalized version of this theorem, stated below, makes only a relatively tame assumption, namely that the vocabulary has finitely many relations of each arity. For example, such vocabularies will arise if we start with some finite vocabulary, and then enrich it by creating a new relation symbol for every quantifier-free formula (modulo equivalence of quantifier-free formulas). In the following theorem, when speaking of a symmetric monoidal closed category, we mean that properties stated in the conclusion of Theorem 3.8, in particular we do not require unique Currying.

Theorem 4.2. *Consider a relational structure $\mathbb{A}$, in which for every $k \in \{0, 1, \ldots\}$ there are finitely many relations of arity k. Then the single-use category over $\mathbb{A}$ is symmetric monoidal closed, with respect to the tensor product $\otimes$.*

The theorem is proved in the same way as Theorem 3.8. The assumption on the vocabulary is used to ensure that in the corresponding game semantics, there are finitely available moves in any given moment, because the vocabulary can only be queried up to the maximal number of atoms in the input, due to the single-use restriction.

Note that this theorem can be applied to any relational structure, including undecidable ones. Clearly, there must be some benefit from assuming that the structure has a decidable first-order theory, which means that there is an algorithm which checks if a first-order sentence is true in the structure. The benefit is that we can check if two morphisms are equal, as expressed in the following theorem.

Theorem 4.3. *Consider a relational structure $\mathbb{A}$, in which for every $k \in \{0, 1, \ldots\}$ there are finitely many relations of arity k. If this structure has a decidable first-order theory, then there is an algorithm for the following problem:*

- **Input.** *Two morphisms, described by expressions using prime functions and combinators.*
- **Question.** *Are they the same morphism?*

The above theorem gives us a reasonable category of single-use functions over a relational structure with a decidable first-order theory. This applies to structures such as Presburger arithmetic, or the real field $(\mathbb{R}, +, \cdot, \leq)$ with the field operations viewed as ternary relations. However, a decidable first-order theory is not the only property needed to ensure that the category is appropriate to automata. If we want to model automata and their decision procedures, we will also need to execute certain fixpoint algorithms, as explained in [3]. This will be illustrated in the next section, where we prove that emptiness for automata, including two-way automata, is decidable under an additional assumption on the structure called *oligomorphism*. This assumption is the standard assumption used to ensure that orbit-finiteness is meaningful.

5. Traced categories and two-way automata

In this section, we show that the single-use category is traced with respect to co-product, and we use this fact to model deterministic two-way automata. We begin by describing the usual trace construction in the category of sets and partial functions. Consider some partial function $f : A + X \to B + X$. For a number $k \in \{0, 1, \ldots\}$ we can define a partial function of type $A \to B$, which is called the *k-th iteration*, as follows by induction on k. The 0-th iteration is completely undefined. For $k > 0$, the k-th iteration is defined as follows. First we apply f to

the input. If the output is undefined or from B, then that is the final output of the k-th iteration. Otherwise, if the output is from X, then we apply the $(k-1)$-st iteration and return that output (which may be undefined). It is easy to see that the iterations are ordered by inclusion, i.e. when viewed as binary relations they form an increasing chain. The limit of this chain, which is a partial function from A to B is defined to be the *trace of* f.

We will now show that the traces exist also in the single-use category, even if we take the general variant from Section 4 that arises from some relational structure. However, we will need to make a certain assumption on this structure. Call a structure $\mathbb{A}$ *oligomorphic* if for every $k \in \{0, 1, \ldots\}$, there are finitely many orbits in $\mathbb{A}^k$ under the action of the automorphism group of $\mathbb{A}$. Oligomorphism is the standard assumption in the theory of sets with atoms [3]. In particular, ensures that the notion of orbit-finite set is meaningful. Examples of oligomorphic structures include: the atoms with equality only, the rational numbers with their linear order, and the Rado graph. Nonexamples include: the integers with their linear order, Presburger arithmetic, and the real field.

The main observation is that for single-use functions over an oligomorphic structure, the trace is achieved in finitely many steps.

Lemma 5.1. *Let $\mathbb{A}$ be an oligomorphic structure, let A, B, X be linear types, and let*

$$f : [\![A + X]\!] \to [\![B + X]\!]$$

be a single-use function over $\mathbb{A}$. There is some k such that the trace of f is equal to $f^{(k)}$.

Since the trace is achieved in finitely many steps, and it is easily seen to be constructed using operations on functions that preserve the single-use condition, the trace is also single-use. (In this section, we use single-use partial functions, which are defined to be single-use functions of type $X \to Y + 1$, where 1 represents the undefined value.) Therefore, the single-use partial functions over an oligomorphic structure form what is called a *traced category*, with respect to $+$.

We now use the trace construction to model deterministic two-way automata. (The idea that traced categories are a natural setting for two-way automata was already noted in [14].) We define a deterministic two-way automaton in the same way as a deterministic one-way automaton, except that the transition function now is a partial function from $\Sigma \otimes (Q + Q)$ to $Q + Q + 1$. The two copies of Q represent entering the letter from the left or right, and the 1 in the output represents acceptance. The function is partial, and thus if we want to model it as a complete function, then there will be another 1 in its output, representing rejection.

Theorem 5.2. *Let $\mathbb{A}$ be an oligomorphic structure with a decidable first-order theory. Then the emptiness problem is decidable for single-use deterministic two-way automata over $\mathbb{A}$.*

The single-use restriction is crucial, since without it the emptiness problem becomes undecidable for all choices of $\mathbb{A}$ where the universe is infinite. The above theorem is the first known example of two-way model that has decidable emptiness and uses atoms that are not the equality atoms.

6. Orbit-finite vector spaces

As a complement to the single-use category described before, we present a second example of a symmetric monoidal closed category that uses atoms. In this example, instead of restricting functions, we generalize them. This generalization is based on the orbit-finitely spanned vector spaces that were introduced in [6]. As mentioned

in the introduction, this section contains no new technical results beyond those from [6].

To explain the intuitive reason behind the usefulness of vector spaces, consider finite (and not orbit-finite) sets. The number of functions from set with n elements to a set with m elements is m^n. However, if we consider linear maps instead of functions without any structure, then we get an exponential improvement, namely instead of m^n we will have $m \cdot n$. This is because a linear map from a vector space of dimension n to a vector space of dimension m can be specified by giving an $m \times n$ matrix. As we show in this section, in the orbit-finite world, the improvement is more pronounced: from infinite to finite.

For the sake of concreteness, all vector spaces will be over the field of rational numbers. However, the choice of the field will not affect the results. The general idea is that the field has no atoms in it, and therefore it has a trivial action of atom automorphisms.

Definition 6.1 (Vector space with atoms)**.** A *vector space with atoms* is a set with atoms, equipped with a vector space structure, such that the vector space structure is equivariant, i.e. vector addition is equivariant, and scalar multiplication $v \mapsto cv$ is equivariant for every field element c.

Example 6.2. For a set X, let us write $\mathsf{Lin}X$ for the vector space that consists of finite linear combinations of elements from X. When X is the set of atoms, the resulting vector space $\mathsf{Lin}\mathbb{A}$ is a vector space with atoms, because it comes equipped with a natural action of atom automorphisms. An example vector in this space is $3 \cdot \text{John} + 2 \cdot \text{Eve} - 5 \cdot \text{Mark}$. ◀

When talking about vector spaces with atoms, one has to be careful about using bases. This is because finding a basis might require choice, and choice is not available in the presence of atoms. For example, consider space $\mathsf{Lin}\mathbb{A}$ from Example 6.2, and its subspace V which contains those vectors where all coefficients sum up to zero. This space is spanned (i.e. generated using linear combinations) by the orbit-finite set which consists of all pairs $a - b$, where a and b are distinct atoms. However, this spanning set is not a basis, since it contains linearly dependent vectors, for example John - Eve and Eve - John. Keeping only one of these vectors would require choice, which cannot be done using equivariant functions, and in fact this space has no basis that is equivariant [6, Example 6]. For this reason, the appropriate notion of finiteness is having an orbit-finite spanning set, which is not necessarily linearly independent. This leads to the following category.

Definition 6.3. The category of orbit-finitely spanned vector spaces is:

(1) The objects are vector spaces with atoms that have an orbit-finite spanning set.
(2) The morphisms are equivariant linear maps.

For the above category, we will use tensor product $\otimes$; this is the usual notion of tensor product for vector spaces. For example, if we take vector spaces $\mathsf{Lin}X$ and $\mathsf{Lin}Y$, i.e. finite linear combinations of orbit-finite sets X and Y respectively, then the resulting tensor product is $\mathsf{Lin}(X \times Y)$. More generally, the tensor product of two orbit-finitely spanned vector spaces is also orbit-finitely spanned [6, Theorem VI.3].

Theorem 6.4. *The category of orbit-finitely spanned vector spaces is symmetric monoidal closed, with respect to the tensor product.*

A corollary of the above theorem is that, in the category of orbit-finitely spanned vector spaces, deterministic automata recognize the same languages as monoids.

This is because the usual translation can be done, with a deterministic automaton of states Q being mapped to a monoid with elements $Q \Rightarrow Q$. This was already observed in [6, Theorem VIII.3].

However, there are two limitations of the orbit-finitely spanned vector spaces.

The first limitation is that the existence of function spaces is dependent on the choice of atoms. Theorem 6.4 works when the atoms have equality only, and it also works when the atoms are equipped with a total order. This is because the dual spaces are orbit-finitely spanned in these cases, as proved in [6, Corollary VI.5]. However, the dual spaces are no longer orbit-finitely spanned for other choices of atoms, such as the Rado graph, see [6, Example 9]. This is in contrast to the single-use category, where the existence of function spaces is independent of the choice of atoms.

A second limitation is that it does not support two-way automata, unlike the single-use category (see Section 5). This is because this category generalizes the orbit-finite category (i.e. it admits a faithful functor from it), and in the orbit-finite category emptiness is undecidable for deterministic two-way automata [17, Theorem 5.3]. This precludes the kind of traced construction that we did in the single-use category. This issue appears already without atoms: the category of finite-dimensional vector spaces is not traced with respect to the sum $+$ of vector spaces.

Appendix A. Game semantics

This section is devoted to the proof of Theorem 3.8. To construct the function space $X \Rightarrow Y$, we use game semantics to identify a certain normal form of programs that compute single-use functions. The presentation in this section is self-contained, and does not assume any knowledge of game semantics. We base our notation on [1].

Let us begin with a brief motivation for why game semantics will be useful.

While it is intuitively clear which functions should be allowed as single-use for simple types such as $\mathbb{A} \to 1 + 1$ or $\mathbb{A} \otimes \mathbb{A} \to \mathbb{A} + \mathbb{A}$, these intuitions start to falter when considering more complex types. How can one show that a function is *not* single-use? If one were to use the definition of single-use functions alone, then one would need to rule out any possibility of constructing the function from the primes using the combinators, including constructions that use composition many times, and with unknown intermediate types.

This is the reason why we consider game semantics. It will allow us to give a more principled description of the intuition that pairs of type $X \otimes Y$ can be used on both coordinates, while pairs of type $X + Y$ can be used on a chosen coordinate only. The idea behind game semantics is to give the description in terms of an interaction between two players. The two players are:

(1) System, who represents the function (we will identify with this player); and
(2) Environment, who supplies inputs and requests outputs of the function.

One of the intuitions behind the setup is that if a type $X \& Y$ appears in the input of the function, then it is the System who can choose to use X or Y, while if the type appears in the output, then it is the Environment who makes the choice. (In this paper, we consider functions of first-order types of the form $X \to Y$, where X and Y are linear types that do not use $\otimes$, and therefore there will be a clear distinction between input and output values.) Before giving a formal definition of game semantics, we give simple example of the interaction.

Example A.1. Consider the two types

$$X \otimes (Y \& Z) \quad \text{and} \quad (X \otimes Y) \& (X \otimes Z).$$

Among the prime functions in Table 2, we find distributivity in the direction $\to$, but not in the direction $\leftarrow$. We explain this asymmetry using the interaction between two players System and Enviroment.

Let us first consider the interaction in the direction $\to$. Player Environment begins be requesting an output. Since this output is of type $(X \otimes Y)\&(X \otimes Z)$, this means that Environment can choose to request either of the two types $X \otimes Y$ and $X \otimes Z$. Suppose that Environment requests $X \otimes Y$. Now player System needs to react, and produce two elements: one of type X and one of type Y. Both can be obtained from the input; for the second one player System can choose how to resolve the input $Y\&Z$ to get the appropriate value.

Consider now the interaction in the opposite direction $\leftarrow$. As we will see, player System will be unable to react to the behavior of player Environment, which will demonstrate that there is no distributivity in this direction. The problem is that player Environment can begin by requesting an element of type X, since the output type is $X \otimes (Y\&Z)$, while still reserving the possibility to request $Y\&Z$ in the future (because the tensor product $\otimes$ means that both output values need to be produced). To produce this element, player System will need choose one of the two coordinates in the input type $(X \otimes Y)\&(X \otimes Z)$, and any of these two choices will be premature, since player Environment can then request the opposite choice in the output type. ◀

As illustrated in the above example, we will use a game to describe the possible behaviours of a function, as modelled by behaviours of player System. The game will be played in an arena, which will arise from the type of the function, and will tell us what are the possibilities for the moves of both players. Here is the outline the plan for the rest of this section.

(1) For every two linear types X and Y, we define an arena, which is a data structure that describes all possible interactions between players Environment and System that can arise when running a single-use functions of type $X \to Y$;
(2) In each arena, we will be interested in the strategies of player System, i.e. the ways in which System reacts to moves of Environment. We will show how such strategies can be interpreted as single-use functions: to a strategy we will assign a single-use function of type $X \to Y$ that is represented by this strategy. This mapping will be partial, i.e. some ill-behaved strategies will not represent any functions.
(3) We will show that the set of strategies in an arena is large enough to represent all single-use functions, but small enough to be orbit-finite.
(4) We will then strengthen this: not only is the set of strategies orbit-finite, but it can be equipped with the structure of a linear type, such that both evaluation and Currying will be single-use functions.

The arenas from the first step of the plan will be defined in two sub-steps. We begin by defining arenas and strategies for functions that do not use the structure of the atoms, i.e. constants and equality tests. This will be a fairly generic definition, almost identical to the classical game semantics for linear logic. Then we will extend the definition to cover the extra structure.

A.1. **Arenas and strategies without constants and equality tests.** We begin with a simpler version of the game semantics, in which the arenas and strategies will describe functions that are not allowed to perform equality tests, and are not allowed to use constants. These strategies will model functions such as the identity function $\mathbb{A} \to \mathbb{A}$, which directly passes its input to its output, but they will not model the equality test $\mathbb{A} \otimes \mathbb{A} \to 1 + 1$, or the constant functions of type $1 \to \mathbb{A}$.

The general idea is to use standard game semantics for linear logic, with an extra feature that we call *register operations*. The register operations will be used to model the way in which atoms are passed from the input to output. For example, in the identity function, Environment will write the input atom into the register, and then player System will read the output atom from that register. The following definition of an arena is based on the definition from [1, p.4], slightly adapted for the context of this paper:

Definition A.2 (Arena)**.** An *arena* consists of:

(1) A set of *moves*, with each move having an assigned owner, who is either "System" or "Environment", and one of three[6] register operations, which are "none", "read", or "write".
(2) A set of plays, which a set of finite sequences of moves that is closed under prefixes, and such that in every play, the owner of the first move is Environment, and then the owners alternate between the two players.

An arena will correspond to a type. The inhabitants of that type, which will be functions if the type is a functional type $X \to Y$, will be described by strategies in the arena. Such a strategy tells us how player System should react to the moves of player Environment. Intuitively speaking, in the case of a functional type, a strategy will say how the function reacts to requests in the output type and values in the input type. We will only be talking about strategies for player System, so from now on, all strategies will be for player System. The following definition corresponds to the definition from [1, p.5]:

Definition A.3 (Strategy)**.** A strategy in an arena is a subset of plays in the arena, which:

(1) is closed under prefixes;
(2) if the strategy contains a play p that ends with a move owned by player System, then it also contains all possible plays in the arena that extend p with one move of player Environment;
(3) if the strategy contains a play p that ends with a move owned by player Environment, then it contains exactly one play in the arena that extends p with one move of player System;
(4) there is some k such that all plays in the strategy have length at most k;
(5) every "read" move is directly preceded by a "write" move (in particular a play cannot begin with "read"), and every "write" move is either the last move, or directly succeeded by a "read" move.

Conditions 2 and 3, which are standard in game semantics, guarantee that the strategy only "ends" when Environment has no moves to play. Let us now comment on the last two conditions, which are not standard.

The fourth condition is motivated by the idea that we study "finite" types, and there will be no need for unbounded computations.

The last condition will be called the *immediate read condition*. It ensures that there is matching between "read" and "write" moves in plays that do not end with write. Since "write" will always be owned by Environment, the immediate read condition will ensure a matching between "write" and "read" moves.

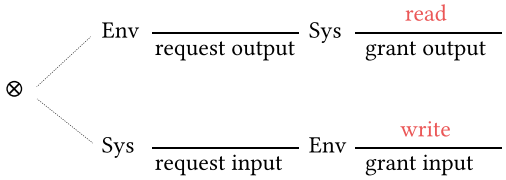
We now show how to associate to each linear type a corresponding arena, and also how to associate an arena to a function type $X \to Y$. This definition will be compositional, i.e. it will arise through operations on arenas that correspond to the

[6] In all arenas that we consider, the "read" moves will be owned by System and the "write" moves will be owned by Environment. Therefore, we could simplify the register operations and have just one, called "read/write", whose status is determined by its owner.

type constructors such as 1 or $\otimes$. The arenas that we will construct so far will not be our final proposal, since the corresponding strategies will not be able to use constants or perform equality tests. This will be fixed in Section A.2, where a more complex arena will be defined for function types. Before giving formal definition, we begin with a simple example of the arena for the type $\mathbb{A} \to \mathbb{A}$.

Example A.4. We define an arena for the type $\mathbb{A} \to \mathbb{A}$. This arena will be rather impoverished, since the only allowed strategy in it will correspond to the identity function. However, this is consistent with the stage that we are at, where we only consider functions that do not use constants or perform equality tests; for such functions the only possibility is the identity function.

The arena will describe the following interaction between the two players: Environment requests an output, then System requests an input, then Environment grants the input, and finally player System grants the output by forwarding the input grant. The arena is shown in the following picture:



The methodology of drawing this picture will become clearer later on, as we define operations on arenas such as $\otimes$. For the moment we describe the arena without caring that it arises as a special case of some general construction. The arena has four moves:

move	owner	register operation
request output	Environment	none
request input	System	none
grant input	Environment	write
grant output	System	read

The set of plays is defined as follows. These are all sequences that begin with a move of player Environment, alternate between players, use each move only once, and have the following condition: "grant output" can only be played after "request output", and likewise for "grant input" and "request input".

A quick inspection of the above definition reveals that the arena has a unique maximal play, where the moves are played in the order from the table, and all other plays are prefixes of this maximal play. Because of the uniqueness of responses, the set of plays is also a strategy. As mentioned at the beginning of this example, the strategy describes the identity function. ◀

We hope that the above example explains some intuitions about how arenas describe types and strategies describe functions. We now give a formal definition. As mentioned before, this definition is compositional: we define arenas for the basic types 1 and $\mathbb{A}$, and then we define operations on arenas that correspond to the type constructors $+$, $\&$, and $\otimes$. We begin with the basic types.

Definition A.5 (Arenas for 1 and $\mathbb{A}$)**.**

(1) The arena for the type 1 is empty: there are no moves and the only play is the empty sequence.
(2) The arena for type $\mathbb{A}$ has two moves, which must be played one after the other: first player Environment makes a move called "request" that has no register operation, and then player System responds with a move called "grant" that has register operation "read".

In the above definition, we only described the behaviour of $\mathbb{A}$ when viewed as an output type. To get the input type, where the players are swapped and read is swapped with write, we will use duality, which is another operation on arenas. This operation, together with other operations that correspond to the type constructors, are defined below.

Definition A.6 (Operations on arenas)**.** Let A and B be arenas. We define the following arenas (see also Figure 1):

$A + B$: The moves in this arena are the disjoint union of the moves of A and B, with inherited owners and register operations, plus three extra moves: "ask" owned by Environment, and "left", "right" owned by System. The plays are defined as follows. Player Environment begins with an ask move, then System responds with a left or right move, and the remaining sequence is a play in the arena A or B, depending on whether System chose left or right.

$A\&B$: The moves in this arena are the disjoint union of the moves of A and B, with inherited owners and register operations, plus three extra moves: "acknowledge" owned by System, and "left", "right" owned by Environment. The plays are defined as follows. Player Environment begins by choosing left or right, then player System responds with an acknowledge move, and the remaining sequence is a play in the arena A or B, depending on whether Environment chose left or right. (This construction differs slightly from the one from [1, Excercise 1.10] – this is because we want to keep it analogous to the construction for $A + B$.)

$A \otimes B$: The moves in this arena are the disjoint union of the moves of A and B, with inherited owners and register operations. A play in this arena is any shuffle of plays in the two arenas A and B. (A shuffle of two words is any word obtained by interleaving them, e.g. shuffles of "abc" and "123" include "a1b23c" and "12ab3c"). By Definition A.5, we require that the owners of the move alternate in the interleaved sequences. (This construction is based on [1, p.7].)

$\bar{A}$: This is called the dual arena of A. The moves and plays are the same as in A, except the owners are swapped, and the "read" and "write" register operations are swapped.

Equipped with the above definitions, we present our first attempt at assigning arenas to types. In the second item of the following definition, we use the name *library-less*, because our final definition of the arena for a function type, as presented in the next section, will be equipped with an extra feature that will be called a library.

Definition A.7. Let X and Y be linear types.

- The *arena for* X is defined by inductively applying the constructions from Definition A.5 and Definition A.6 according to the structure of the type.
- The *library-less arena for* $X \to Y$ is defined to be (dual of arena of X) $\otimes$ (arena of Y).

As discussed previously, our notion of arenas does not yet take into account the structure of the atoms, i.e. the constants and equality tests. This will be fixed in the next section, by modifying the second item in the above definition. On the other hand, the arenas from that first item in the above definition, for linear types without function types, are already in their final form.

In principle the construction from the second item in the above definition can be nested, and thus used to assign arenas to higher order types that can nest $\to$ with the other type constructors. This is how it is usually done in linear logic. However,

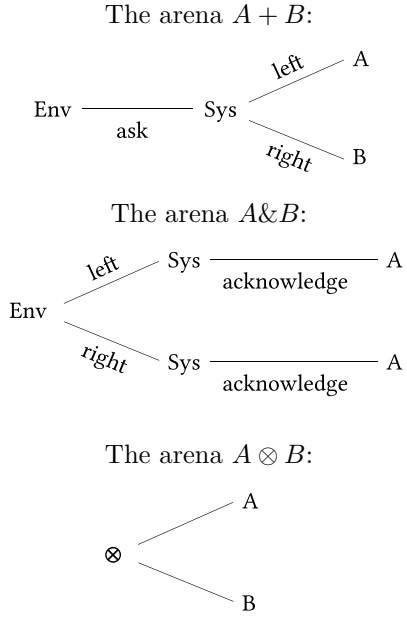


FIGURE 1. Pictures of the operations on arenas. The picture for $\otimes$ is less useful than previous two, since the root node of the tree is not a player, but a node labelled by $\otimes$. The intuition is that the game is played in parallel on both arenas, and therefore a position in it can be visualized as a pair of positions in the two arenas.

the construction that we will describe in the next section will be less amenable to nesting, and will use it only to describe functions between types that do not use $\to$.

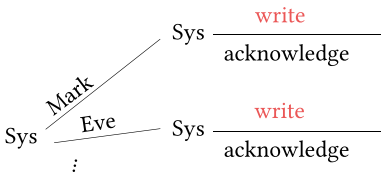
Example A.8. The arena for the identity type $\mathbb{A} \to \mathbb{A}$ is the same as the arena from Example A.4.

A.2. **Arenas and strategies with constants and equality tests.** In the Section A.1, we described arenas for functions that did not use the structure of the atoms, i.e. constants and equality tests. We now show how these arenas can be extended to cover this structure. The general idea is to equip the arenas with an extra part, which we call the *library*, that describes the allowed operations on the atoms. (The library as we present it here only contains functions for equality and constants, but in the proof of Theorem 4.2, we can use a library that has other relations beyond equality.)

Definition A.9 (The library arena)**.** The library arena and its parts are defined as follows, see Figure 2 for pictures.

(1) The *constant choice arena* is the following arena $\mathbb{A} + 1$ moves: first player System chooses an atom, then player Environment plays move with register operation "write".
(2) The *equality test arena* is an arena which the plays are as follows:
   (a) first player System plays a move with register operation "read";
   (b) then player Environment plays an move with no register operation;
   (c) then player System plays a move with register operation "read";
   (d) then player Environment plays one of two moves, called $=$ and $\neq$, with no register operation.

The constant choice arena:



The equality test arena:

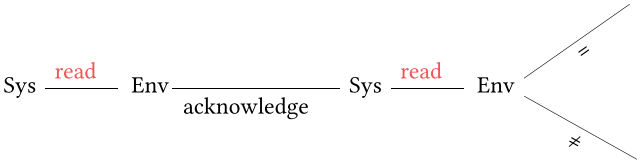


FIGURE 2. Pictures of the library arenas. We use the convention that the register operations are in red, and the names of the moves, which have no other role than to distinguish them, are in black. Note that the first move in this arena is owned by System, and we assume in Definition A.2 that the first move is owned by Environment. This is because this arena, like all arenas in Definition A.9, is not intended to be a stand-alone arena, but only as part of the bigger arena from Definition A.10 where the first move is indeed owned by Environment.

(3) The *library arena* is defined to be an arena that is obtained by applying $\otimes$ to infinitely many copies of the constant choice arena and infinitely many copies of the equality test arena.

The library arena is infinite. Taking the tensor product of infinitely many copies of the two arenas ensures that the library arena satisfies the following property, which corresponds to the ! operation from linear logic:

(A.1)
$$\text{library arena} \quad \equiv \quad (\text{constant choice arena}) \otimes (\text{equality test arena}) \otimes (\text{library arena}).$$

In the above, $\equiv$ refers to isomorphism of arenas, which is defined in the natural way: this is a bijection between the moves, which is consistent with the owners, register operations and plays in the expected way. Another property is that the library arena is isomorphic to a tensor product of itself:

(A.2)
$$\text{library arena} \quad \equiv \quad \text{library arena} \otimes \text{library arena}.$$

We are now ready to give the final definition of arenas for functions between linear types, which takes into account the structure of the atoms.

Definition A.10 (Arena for a function type)**.** For linear types X and Y, the arena of $X \to Y$ is

$$(\text{library arena}) \otimes (\text{dual arena of } X) \otimes (\text{arena of } Y).$$

This completes the game semantics of linear types and functions between them. We do not intend to give game semantics for higher order types, such as functions on functions etc. As a result, we will only be using the dual once, namely for the arena of the input type. Also, note that the read/write operations will be used in

a restricted way, as announced in Footnote 6, namely that the "read" moves will be owned by System and the "write" moves will be owned by Environment. This is because the library arena has this property, the arena for $\mathbb{A}$ has this property, and all operations on arenas that we have defined preserve this property.

A.2.1. *Composition of strategies.* One of the main points about strategies in game semantics is that they can be composed. The usual construction for the function type $X \to Y$ is to take the tensor product of the dual arena for X and the arena for Y. Our construction is a bit more involved, since the arena that we use has a copy of the library arena. We now explain how to compose strategies in a way that accounts for the library arena.

We begin by describing the usual construction for composing strategies, which we call *shuffling and hiding*, see [1, p.12], with a minor adaptation to our setting that has register operations.

Definition A.11 (Shuffling and hiding)**.** Let A, B, C be arenas, and consider two strategies

$$\sigma_1 \text{ in the arena } A \otimes \bar{B} \qquad \sigma_2 \text{ in the arena } B \otimes C.$$

The shuffling and hiding strategy for σ_1 and σ_2, which is a strategy in the arena $A \otimes C$, is defined to be the set of plays p such that there exist plays $p_1 \in \sigma_1$ and $p_2 \in \sigma_2$ with the following properties:

- the play p satisfies the immediate read condition;
- the following two sequences of moves are equal:
  (1) the subsequence of moves in p_1 that are from the arena $\bar{B}$;
  (2) the subsequence of moves in p_2 that are from the arena B.

Note that the set of moves in the arenas $\bar{B}$ is the same as the set of moves in the arena B (the owners and register operations are changed), and therefore the subsequences in items 1 and 2 above can be meaningfully compared. The following lemma, whose straightforward proof is left to the reader, shows that the above definition is well-formed.

Lemma A.12. *The strategy described in Definition A.11 is a valid strategy.*

We can now use shuffling and hiding to compose strategies in the arena for a function type that were defined in Definition A.10. Consider three linear types X, Y, Z, and two strategies, in the arenas for $X \to Y$ and $Y \to Z$, respectively. By unfolding the definition of arenas from Definition A.10, these are strategies in the arenas

$$(\text{library arena}) \otimes \overline{\text{arena for } X} \otimes \text{arena for } Y \qquad \text{the arena for } X \to Y, \text{ and}$$
$$(\text{library arena}) \otimes \overline{\text{arena for } Y} \otimes \text{arena for } Z \qquad \text{the arena for } Y \to Z.$$

Using the construction from Definition A.11, we can combine them into a single strategy in

$$(\text{library arena}) \otimes (\text{library arena}) \otimes \overline{\text{arena for } X} \otimes \text{arena for } Z.$$

Since the library arena is isomorphic to a tensor product of two copies of itself, the above strategy gives us a strategy in the arena

$$(\text{library arena}) \otimes \overline{\text{arena for } X} \otimes \text{arena for } Z \qquad \text{the arena for } X \to Z.$$

This strategy is defined to be the *composition* of the original two strategies. We write $\sigma; \tau$ for this composition. Thanks to [1, Proposition 1.2], we know that the the usual composition of library-free strategies is associative. It follows that our composition of library strategies is associative up to isomoprhism.

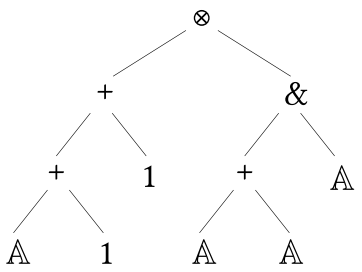


FIGURE 3. The syntax tree of the type $((\mathbb{A}+1)+1)\otimes((\mathbb{A}+\mathbb{A})\&\mathbb{A})$.

A.3. **Strategies as single-use functions.** In this section, we explain how a strategy in a function type $X \to Y$ can be interpreted as a single-use function from the set $[\![X]\!]$ to the set $[\![Y]\!]$. This interpretation is partial, since some strategies will be considered inconsistent, and will not represent any function. We begin with an example which explains how a strategy might be inconsistent.

Example A.13 (Inconsistent strategy)**.** Consider a strategy in the arena for $1 \to \mathbb{A}\otimes\mathbb{A}$. We would like to think of this strategy of producing a pair of atoms. However, an inconsistent strategy for player System could use different constants from the constant choice arena depending on the order in which Environment requests the two output atoms in $\mathbb{A}\otimes\mathbb{A}$. For example, the output could be (John, Mark) for one order, and (Eve, Mary) for some other order. ◀

Below, we will formalize the intuition from the above example. We begin by developing some notation for describing plays.

A.3.1. *Some notation for describing plays.* Consider a move in the arena associated to a function type $X \to Y$. There are three kinds of moves here: moves from the library, moves from X, and moves from Y. Depending on where the moves come from, we will associate to each move an *origin* as follows.

- In the case of moves which come from the library, define the origin to be the copy of the constant choice arena or the equality test arena from which the move is taken.
- In the case of moves which come from either X or Y, define the origin to be the corresponding node in the syntax tree of X or Y. Here, the syntax tree of a type is defined in the obvious way, see Figure 3.

Consider a play in the arena for $X \to Y$. Define a *library call* to be a subsequence of the play that is obtained by choosing some copy of the constant choice arena or the equality test arena, and then returning all moves in the play that originate from this copy. A library call will have at most two moves if it originates from some copy of the constant choice arena (we use the name *constant call* for such library calls), and at most four moves if it originates from some copy of the equality test arena (we use the name *equality call* for such library calls). For each constant call, there is an associated at atom, which is the first move in the call.

Consider a read move in the play. By the immediate read condition, this move is directly preceded by a write move. The write move will either originate from a leaf labelled by $\mathbb{A}$ in the syntax tree of the input type X, or it will originate from some constant call. In the latter case, it will have a corresponding atom. Therefore, to each read move in a play we can associate either some leaf labelled by $\mathbb{A}$ in the syntax tree of the input type, or some individual atom constant. The associated object will be called the *read value* of the read move.

Equipped with the above notation, we are ready to define which strategies can be interpreted as values or functions.

A.3.2. *Strategies in $1 \to X$ as values in $[\![X]\!]$.* We begin by describing strategies which represent values in the underlying set $[\![X]\!]$ of a linear type X, and then we lift this interpretation to functions. To represent such values, we use strategies not in the arena for X, but in the arena for $1 \to X$. This is because the second arena contains the library, which is needed to produce atom constants.

Let X be a linear type, and consider a strategy σ in the arena for $1 \to X$. In order to represent some value, this strategy must satisfy certain conditions that avoid the bad behaviour from Example A.13.

Definition A.14 (Strategy that represents a value)**.** Let X be a linear type and consider a strategy in the arena for $1 \to X$. We say that the strategy uses a node from the syntax tree of X if at least one play in the strategy uses a move that originates from this node. We say that the strategy *represents a value* if it satisfies the following conditions:

(1) Consider a node in the syntax tree of X that is labelled by co-product $+$. Then at most one child of this node is used by the strategy.
(2) Consider a node in the syntax tree of X that is labelled by $\mathbb{A}$. Then there is some atom a such that for every play in the strategy, if the play contains a read move that originates from the node, then the read value of this move is a.

For a strategy that satisfies the two conditions in the above definition, we can also define the value $x \in [\![X]\!]$ that it represents. This is done as follows. For every node in the syntax tree of X which is used by the strategy, we assign a value, which is called the value of the strategy at the node. This value at a node is in the set $[\![Y]\!]$, where Y is the linear type that corresponds to the subtree of the node. The definition of this value at a given node is defined using a straightforward induction on the size of its subtree:

(1) the value at a leaf labelled by 1 is the unique element of $[\![1]\!]$;
(2) the value at a leaf labelled by $\mathbb{A}$ is the atom from the second item in Definition A.14;
(3) the value at a node labelled by $\otimes$ or $\&$ is the pair of values at the children;
(4) the value at a node labelled by $+$ is the value at the unique child in the first item in Definition A.14.

The above definition is well-formed, since one can easily see from the definition of strategies that if a node labelled by $+$ is used by the strategy, then at least one of its children is used, and if a node labelled by $\otimes$ or $\&$ is used, then both of its children are used. Finally, the value represented by a strategy is defined to be the value that is represented at the root node of the syntax tree.

One can easily see that every value in the set $[\![X]\!]$ is represented by at least one strategy. This will also follow from a more general result, Lemma A.16 below.

A.3.3. *Strategies in $X \to Y$ as single-use functions.* We now move to the main point of this section, which is to show how to interpret a strategy in the arena for $X \to Y$ as a single-use function of type $X \to Y$. This is done by using the definition from for values in the previous section, and then lifting it to functions by using composition of strategies that was defined in Section A.2.1.

Definition A.15. Let X and Y be linear types, and let σ be a strategy in the arena $X \to Y$. We say that σ *represents* a function

$$f : [\![X]\!] \to [\![Y]\!]$$

if for every $x \in [\![X]\!]$, if x is the value represented by some strategy τ in the arena for $1 \to X$, then the composition strategy $\tau; \sigma$ in the arena for $1 \to Y$ represents a value, and this value is equal to $f(x)$.

Since each strategy in $1 \to X$ or $1 \to Y$ represents at most one value, it follows that a strategy can represent at most one function. This function is called the *function represented by the strategy*. We are now ready to state the main result of this section, which is that every single-use function is represented by at least one strategy.

Lemma A.16. *Let X and Y be linear types, then every single-use function of type $X \to Y$ is represented by at least one strategy.*

Proof. It suffices to show that all single-use prime functions can be represented as strategies, and to lift the combinators $\circ$, $\otimes$, $\times$, & from functions to strategies in a way that preserves the semantics. Let us start with the combinators:

$f; g$: We compose the corresponding strategies. The correctness of this construction is witnessed by the following claim.

> **Claim A.17.** *Let X, Y, Z be linear types, and let σ_1 and σ_2 be strategies in the arenas for $X \to Y$ and $Y \to Z$, respectively. If both of these strategies represent functions, then $\sigma_1; \sigma_2$ also represents a function, and this function is equal to the composition of the functions represented by σ_1 and σ_2.*
>
> *Proof.* Let τ be some value consistent strategy in the arena for $1 \to X$. By associativity of strategy composition, we know that the strategies
>
> $$\tau; (\sigma_1; \sigma_2) \quad \text{and} \quad (\tau; \sigma_1); \sigma_2$$
>
> are equal, up to an isomorphism of the library arena. Since such an isomorphism does not affect value consistency or the represented value, either both or none are value consistent, and if they are value consistent, then they represent the same value. The right strategy is value consistent by the assumption of the lemma, and therefore the same is true for the left strategy, and the represented values are equal. □

$f_1 + f_2$: Suppose that functions f_1 and f_2 are represented by strategies σ_1 and σ_2. We join these two strategies into a new strategy σ as follows. Player Environment begins, and the only move available is to "ask" for one of the two disjuncts of the output type, which is the co-product of the output types of f_1 and f_2. Player System responds with a corresponding question on the input type. The only choice for player Environment is to select either "left" or "right" in the input type. Player System forwards this choice to the output type, and then uses the corresponding strategy. Every play in this strategy has the following form: it begins with four moves that select a disjunct, and then it continues with a play in the strategy σ_1 or σ_2, depending on the appropriate disjunct. From this property, one can show that the strategy represents the function $f_1 + f_2$.

$f_1 \& f_2$: The construction is defined analogously to the one for $f_1 + f_2$.

$f_1 \otimes f_2$: Again, we use an analogous construction, in which player System plays in parallel the two strategies representing the functions f_1 and f_2. The corresponding property, which ensures that the strategy represents the function $f_1 \otimes f_2$, is that each play can be split into a shuffle of two plays, one for the strategy representing f_1 and one for the strategy representing f_2.

Let us now deal with prime functions. For the sake of brevity we only show the strategies for selected prime functions. The remaining prime functions can be implemented without using the library, and are well known strategies corresponding to tautologies of affine logic:

Equality test of type $\mathbb{A} \otimes \mathbb{A} \to 1 + 1$**:** : For this function, System uses the library function for comparing atoms and returns the result. The library is designed to implement this function, so the construction is straightforward, but we describe it here for the convenience of the reader:
- Environment begins by playing "ask" in $1 + 1$, this is the only choice
- System plays "request" in the left $\mathbb{A}$.
- Environment replies with "grant" which is a write move, this is the only choice.
- System plays the read move in atoms equality library function.
- Environment acknowledges, this is the only choice.
- System plays "request" in the right $\mathbb{A}$.
- Environment plays "grant", this is the only choice.
- System plays the read move in the atoms equality function.
- Environment replies with either $=$ or $\neq$ in the library, these are the only choices.
- System forwards this choice to the output type.

Constant function of type $1 \to \mathbb{A}$**:** As in the previous item, this function is implemented by using the library, in this case the constant choice part of the library.

Projection of type $X \otimes Y \to X$**:** System uses the shuffling and hiding strategy that was described in Section A.2.1.

Distributivity of type $X \otimes (Y \& Z) \to (X \otimes Y) \& (X \otimes Z)$**:** The strategy from Example A.1.

□

One could also show that all represented functions are single-use. We omit this proof, since we will prove a stronger result in the next section.

A.4. **The set of strategies as a linear type.** The purpose of this section is to show that the set of strategies in an arena for a function type $X \to Y$ can be described using some linear type, and furthermore the relevant operations on strategies, such as application and Currying, can be performed in a single-use way.

Before we do this we introudce the idea of a *partially applied arena*. Let A be an arena, and m a move. We define the partially applied arena $m^{-1}A$ to be set of sequences that follow m in A, i.e:

$$m^{-1}A = \{s \mid ms \in A\}.$$

The partially applied arena also remembers two bits of information about its context: (a) wheather the current move belongs to the Environment or to the System (b) wheter the previous move was a write move (so that we know if the next move has to be a write move). We finish our discussion with the observation that if moves a, b belong to $\mathbb{A}$ (i.e. they are constant requests from the library), then $a^{-1}A = b^{-1}A$. For this reason we define the notation $\mathbb{A}^{-1}A$ for partially applying any of the atomic moves.

We are now ready the type for storing k-bounded strategies. The following definition inputs an explicit bound k for the length of the plays, but later on in this section will show that such a bound can be derived from the types by using an analysis of strategies.

Definition A.18. For a (partially applied) arena A, we define the type $\mathrm{Strat}(A, k)$ of k-bounded strategies on A using the following induction:

- $k = 0$. This type is 1 if the Environment owns A and has no moves to play. Otherwise, this type is $\emptyset$. (Observe that $\emptyset$ is not a valid type in our language, so in the inductive step we are going to handle it explicitly – the

main idea is that if its used with + then it is going to be ignored, and if it is used with & then the entire type is going to be $\emptyset$.)

- $k+1$. This type is defined differently, depending on whether Environment or System owns A.
  - Environment owns A. Let $m_1, \ldots, m_n$ be the set of possible first moves. (This set is finite.) The type is defined to be
    $$\mathrm{Strat}(m_1^{-1}A, k) \quad \& \quad \cdots \quad \& \quad \mathrm{Strat}(m_n^{-1}A, k).$$
    Let us briefly mention how to handle edge cases: If at least one $\mathrm{Strat}(m_n^{-1}A, k)$ is equal to $\varnothing$, then the entire type is equal to $\varnothing$. If there are no valid moves m_i, then the entire type is equal to 1.
  - System owns A. Now we also consider two options, depending on whether the previous move by Environment was a write move. If it was a write move, then let $m_1, \ldots, m_n$ be the current set of possible read moves that are not the first moves in any library component. (This set is finite). Then our type is defined as follows:
    $$\mathrm{Strat}(m_1^{-1}A, k) + \cdots + \mathrm{Strat}(m_n^{-1}A, k) + \underbrace{\mathrm{Strat}(\mathrm{eq}^{-1}A, k)}_{\substack{\text{Start a new call to}\\ \text{atoms comparing}\\ \text{library function}}}.$$
    The other case is when the previous move was not write. Then let $m_1, \ldots, m_n$ be the set of available moves that are not read moves. (This set is finite again.) The strategy type is defined as:
    $$\mathrm{Strat}(m_1^{-1}A, k) + \ldots + \mathrm{Strat}(m_n^{-1}A, k) + \underbrace{\mathbb{A} \otimes \mathrm{Strat}(\mathbb{A}^{-1}A, k)}_{\substack{\text{Start a new call to}\\ \text{atomic constants}\\ \text{library function}}}.$$
    Again, let us mention the edge cases: If some $\mathrm{Strat}(m^{-1}A, k)$ is empty (including the cases where m is the new library move, i.e. $\mathbb{A}$ or eq) then we simply skip it. If all $\mathrm{Strat}(m_i^{-1}A, k)$ are empty, then the entire type is empty.

The intuition behind this definition, which was already discussed in the main body of the paper, is as follows. If the current move belongs to Environment, then the strategy for System needs to be prepared for every possible move by the environment. However, eventually Environment will play only one move, so only one of those paths will materialize. This behaviour corresponds exactly to the connective &, and is the reason why it is used to make our single-use category symmetric monoidal closed. When the system is about to move, then it needs to pick one of the possible moves. This corresponds to the behaviour of the connective +.

In order to find a linear type that could represent all functions in $X \to Y$, we are going to show that for every linear types X and Y there is a bound k, such that every single use function $f : X \to Y$ can be represented as a k-bounded strategy, i.e. as $\sigma \in \mathrm{Strat}(X \to Y, k)$. Before we do this, let us point out some potential problems.

The main problem is that (as long as $Y \not\approx 1$), there is no universal bound k for strategies in the arena for $X \to Y$ (even though each strategy is bounded by some k). The reason is that System might ask an arbitrary number of irrelevant questions, i.e. questions that use the equality test on two constants. Interestingly, the irrelevant questions might appear as a result of compositions of strategies:

Example A.19 (Irrelevant questions from composition)**.** Consider the following two functions:

(1) the function of type $1 \to \mathbb{A} \otimes \mathbb{A}$ that outputs the pair (Mark, John) for its unique input;
(2) the equality test of type $\mathbb{A} \otimes \mathbb{A} \to 1 + 1$.

Like all single-use functions, the above two functions have natural representations that do not ask irrelevant questions. However, if we apply the composition construction from Section A.2.1 to these two strategies, we will get a strategy that asks the irrelevant question of whether Mark and John are equal. ◀

The idea of irrelevant questions captures the intuition of why strategies might be arbitrarily long, but it is not the only reason for this. Another reason is that System might use constants to start an arbitrary number of parallel calls to the equality test library function, never completing any of them. This is why we introduce the following definition:

Definition A.20 (Well-formed strategy)**.** We say that a strategy is *well-formed* if it never starts a new call to the equality test library function, by passing a constant as its first argument.

Well-formed strategies are bounded, as shown by the following lemma:

Lemma A.21. *For every linear types X and Y there is a universal bound k, such that all well-formed strategies in $X \to Y$ are bounded by k.*

Proof. We start by defining the dimension of a type to be the maximal number of $\mathbb{A}$ that the type can store at the same time. This is defined inductively, where $\dim(1) = 0$, $\dim(\mathbb{A}) = 1$, and the dimension of complex types is computed as follows:

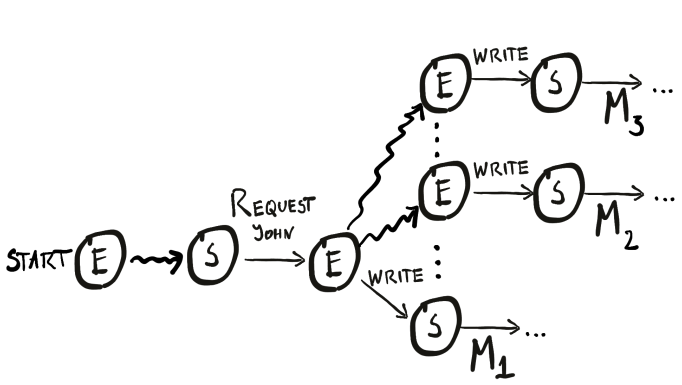
$$\dim(X + Y) = \dim(X \& Y) = \max(\dim(X), \dim(Y)) \quad \text{and} \quad \dim(X \otimes Y) = \dim(X) + \dim(Y)$$

This is because in case of $X \otimes Y$ both X and Y exist at the same times, and in case of $X + Y$ and $X \& Y$ the values X or Y only one of them can materialize.

The key observation is that in a well-formed strategy, every constant granted by the environment is either compared with an atom from input, or moved to the output. It follows that in a well-formed strategy, the system can ask for at most $\dim(X)+\dim(Y)$ constants (because, due to the immediate read rule, each constant has to be used). It is now not hard to see that the lengths of well-formed strategies in $X \to Y$ are bounded by some k that depends only on the types X and Y. □

As illustrated by Example A.19, well-formedness is not preserved under composition of strategies. For this reason, we need that every strategy that is not well-formed can be turned into a well-formed one in a way that preserves its semantics (later we will also show that this can be done in a single-use way). Before we do this, we would like to assume a weaker property of the possibly non-well-formed strategy of the input:

Definition A.22 (Read-consistent strategy)**.** A strategy is called *read-consistent* if, after requesting an atomic value from the environment, it uses the same move to consume this value in all the branches. This request can be made either through a move in the constant library component or a "request" move in an $\mathbb{A}$ on the input. Here is a visual representation:



(The squiggly lines represent more than one move). Here System requests the atom John from the environment. Environment might respond immediately with write (as in the bottom branch) or it might wait some time (as in the middle or top branch). The read-consistency property requires that the system uses the same move to read the value John in all of those branches (i.e. $M_1 = M_2 = M_3$).

The advantage of read-consistency over well-formedness is that it is preserved under compositions of strategies. (One can show this directly from A.11 – the key idea is to compare the system's response to the immediate write, with the system's response to every other possible write). Using this observation, we can show that read-consistent strategies are complete:

Lemma A.23. *For every single use function $f : X \to Y$, there is a read-consistent strategy in the arena $X \to Y$ that represents it.*

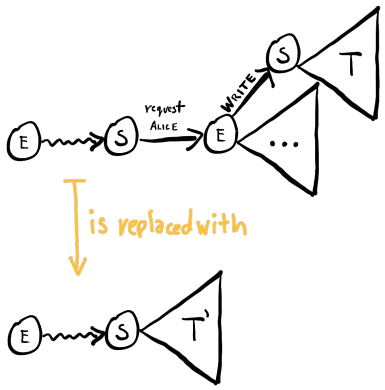
Proof. The proof is analogous to the proof of Lemma A.16. As well-formedness is preserved under the combinators $\circ$, $\otimes$, &, $+$. (The proof for $\otimes$, &, and $+$ is immediate.) Moreover, it is not hard to see that all prime functions can be realized by read-consistent strategies. □

Next, let us show that every read-consistent strategy can be made well-formed in a way that preserves its semantics (thus proving completeness of well-formed strategies).

Lemma A.24. *For every read-consistent strategy in the arena for $X \to Y$, there is a well-formed strategy in the same arena that represents the same function.*

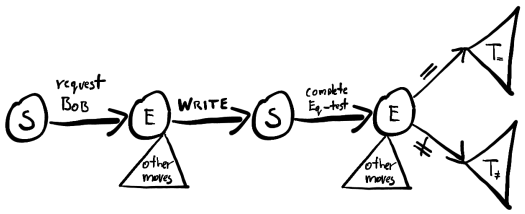
Proof. Consider a system's request for a constant, in some branch of σ. We say that this request is *offending* if System's intention (according to read-consistenty) is to use the constant to start a new call to the equality test library function. Let us now show how to remove an offending request from a strategy in a way that does not change the represented function (without introducing any new offending requests). This is enough to prove the lemma, because we can repeat this construction for all offending moves. (And thanks to read-consistency, we know that if a strategy contains no offending moves, then it is well-formed.)

Let "request Alice" be the offending move. Here is how we remove it:

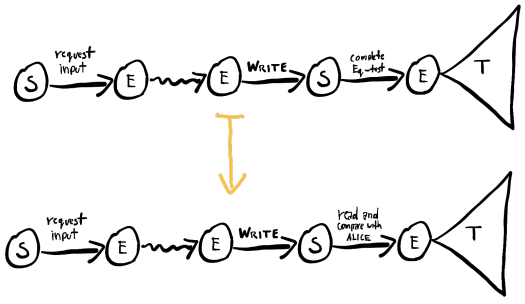


In the above picture, T' represents the strategy obtained by inlining the atom Alice in T. Specifically we need to remove all moves that are used to complete the equality test library function that was initiated with the atom Alice. In order to do this, we identify all atom requests by the system, whose value the system intends to compare with Alice. (In each of T there can be at most one such move.) These could be of two kinds.

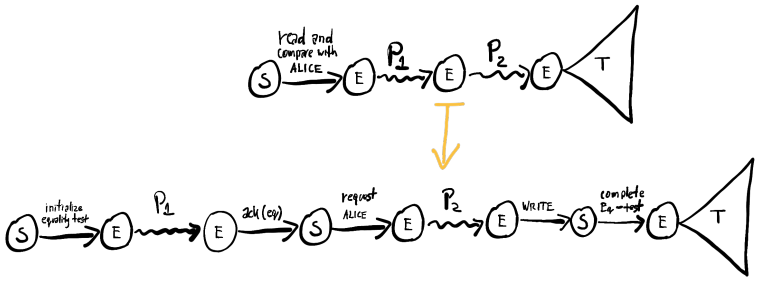
(1) The first kind is a request for another constant. We deal with it in the following way: We ask what would happen if the Environment immediately returned a write move. In this case, we know that System would respond with the completion of the equality test. Then, we consider what would happen when Environment immediately returns the result of the comparison – either $=$ or $\neq$ and replace the entire subtree with the appropriate move. For example, we replace the following subtree with $T_{\neq}$:



(2) We now consider the second kind of offending move, when System requests an atom from the input. In this case, we look at the entire subtree and replace each completion to equality call with a single-move "read and compare with Alice", which represents a new fast-track call to the equality test library function, that reads an atom and compares it with a constant (we will show how to get rid of this in the next paragraph).



This leaves us with showing how to get rid of the "read and compare with Alice" move. We replace each such move with a usual equality test initialization (a read move). Then, as long as Environment does not play the 'acknowledge' move from the equality test, we play as in the original strategy. As soon as the Environment plays 'acknowledge', we play "request Alice" to the constant library component. Then, until the environment plays "write Alice", we continue to play as in the original strategy. Once the environment plays "write Alice", we play the "complete equality test" move, and then we continue to play according to the original strategy. (This is possible, because the responses to "complete equality test", are the same as the responses to "read and compare with Alice"). Here is a picture:



This completes the proof of Lemma A.24. □

By Lemmas A.24 and A.21 we know that every function $X \to Y$ can be represented by a strategy bounded by k that depends only on X and Y. Thanks to this observation we are finally ready to define the linear type for function spaces, which is the main subject of Theorem 3.8.

Definition A.25 (The function space type)**.** Let X and Y be linear types. We define $X \Rightarrow Y$ to be equal to be the type $\mathrm{Strat}(X \to Y, k)$ of k-bounded strategies, where k is the bound from Lemma A.21.

It remains to define evaluation and Currying as single-use functions. To do this, let us introduce first a couple of auxiliary functions. The following lemma, whose straightforward proof we omit, shows that we can transform a strategy into a value.

Lemma A.26. *For every linear X, there is a single-use function of type $(1 \Rightarrow X) \to X$ that returns the value represented by a strategy (see Section A.3.2) if it exists, and otherwise returns some unspecified value.*

The unspecified value in the above lemma is used to ensure that the function is total, since our single-use functions are total. We can simply fix some value $x \in [\![X]\!]$, which is constructed using constants, and return this value for all strategies that do not represent any value. We will use this approach to representing partial functions later in this section as well.

Next, we show that we can compose strategies in a single-use way.

Lemma A.27. *Let $X \Rightarrow Y$ and $Y \Rightarrow Z$ be linear types, then there is a single use function:*

$$\mathit{compose} : (X \Rightarrow Y) \otimes (Y \Rightarrow Z) \to (X \Rightarrow Z)$$

such that if σ_1 and σ_2 are strategies that represent functions f_1 and f_2 respectively, then $\mathit{compose}(\sigma_1, \sigma_2)$ represents the composition of f_1 and f_2.

Proof. The proof has two steps:

- First, we show that for every k, l there is a function $\mathrm{Strat}(X \to Y, k) \otimes \mathrm{Strat}(Y \to Z, k) \to \mathrm{Strat}(X \to Z, k + l)$, that computes the composition of the strategies (according to the A.11). This function can be implemented (by induction on $k+l$) by simulating the two strategies and passing around the moves in Y. The construction will turn out to be single-use thanks to that the hiding operation is an injection on the shuffled strings see [1, Covering Lemma].
- Next we show that for every l there is a single-use function $\mathrm{Strat}(X \to Y, l) \to \mathrm{Strat}(X \to Y, k)$, where k is the constant from Lemma A.21. The function works by removing the offending requests from the strategy, as described in the proof of Lemma A.24. The key observation that makes this construction single-use is that we can use the distributivities of $+$ and $\&$ to push atoms from the node of a tree to all of its children:

$$\mathbb{A} \otimes (T_1 \& \ldots \& T_n) \to \mathbb{A} \otimes T_1 \& \ldots \& \mathbb{A} \otimes T_n \quad \text{and} \quad \mathbb{A} \otimes (T_1 + \ldots + T_n) \to \mathbb{A} \otimes T_1 + \ldots + \mathbb{A} \otimes T_n$$

□

The final helper function transforms values to strategies. This is done in the following lemma, whose straightforward proof we omit.

Lemma A.28. *There is a single-use function of type $X \to (1 \Rightarrow X)$, that returns a strategy whose semantics is equal to its input.*

We are now ready to define Currying and evaluation, and thus complete the proof of Theorem 3.8. We begin with the evaluation function, which is defined as follows:

$$\begin{array}{c} X \otimes (X \Rightarrow Y) \\ \downarrow \text{function from Lemma A.28 on first coordinate} \\ (1 \Rightarrow X) \otimes (X \Rightarrow Y) \\ \downarrow \text{composing function from Lemma A.27} \\ 1 \Rightarrow Y \\ \downarrow \text{function from Lemma A.26} \\ Y \end{array}$$

Now we consider the Currying of a single-use function

$$f : Z \otimes X \to Y.$$

By Lemma A.24 we know that this single-use function is represented by some element $\hat{f}$ of the set represented by the type $Z \otimes (X \Rightarrow Y)$. We use this element to define Currying of f as follows:

$$\begin{array}{c} Z \\ \downarrow \text{extend the input with } \hat{f} \\ Z \otimes (Z \otimes X \Rightarrow Y) \\ \downarrow \text{apply function from Lemma A.28 to first argument} \\ (1 \Rightarrow Z) \otimes (Z \otimes X \Rightarrow Y) \\ \downarrow \text{lift first argument to function that copies argument from } X \\ (X \Rightarrow X \otimes Z) \otimes (Z \otimes X \Rightarrow Y) \\ \downarrow \text{composing function from Lemma A.27} \\ X \Rightarrow Y \end{array}$$

These functions are single-use, as compositions of single-use functions. They also satisfy the required properties in a symmetric monoidal closed category, which can easily be verified using the properties proved in the lemmas that were used to define the functions. This completes the proof of Theorem 3.8.

Appendix B. A quotient construction

A drawback of Theorem 3.8 is that the function space $V \Rightarrow W$ can contain different representations of the same function; this will mean that Currying is not unique. To overcome this issue, we use a simple quotient construction. Define a *partial equivalence relation* to be a relation that is symmetric and transitive, but not necessarily reflexive. This is the same as (complete) equivalence relation on some subset. We will use a partial equivalence on the function space $X \Rightarrow Y$ to: (1) remove objects that do not represent any morphism; (2) identify two objects if they represent the same morphism. After such a quotient, the function space will have unique representations for functions.

Definition B.1. The *quotiented single-use category* is:

- Objects are pairs (linear type X, equivariant partial equivalence relation on $[\![X]\!]$);
- Morphisms between objects $(X, \sim_X)$ and $(Y, \sim_Y)$ are single-use functions from $[\![X]\!]$ to $[\![Y]\!]$ such that the domain of the function is contained in the domain of $\sim_X$, and equivalent inputs are mapped to equivalent outputs.

The quotiented single-use category is also equipped with a tensor product $\otimes$ on its objects.

Theorem B.2. *The quotiented single-use category, equipped with the tensor product $\otimes$, is a monoidal closed category, i.e. it satisfies the conclusions of Theorem 3.8, but, furthermore, the morphism h is unique.*

Appendix C. Beyond equality

As mentioned in the main body of the paper, the proof of Theorem 3.8 extends without any difficulty to the case of relational structures. We only prove the decidability result from Theorem 4.3.

Proof of Theorem 4.3. When defining the category of single-use functions, we viewed it as a restriction of a larger category, in the morphisms were equivariant functions. We begin by describing the generalization of this larger category to the case where the parameter $\mathbb{A}$ is some relational structure. The idea is to use first-order definable functions as the general analogue of equivariant functions.

Definition C.1 (Category of first-order definable functions)**.** Let $\mathbb{A}$ be a relational structure. The *category of first-order definable functions over* $\mathbb{A}$ is defined as follows.

- The objects are polynomial sets over $\mathbb{A}$, i.e. finite disjoint unions of powers of $\mathbb{A}$.
- The morphisms are the first-order definable functions. Here, a function
$$f : \mathbb{A}^{n_1} + \cdots + \mathbb{A}^{n_k} \to \mathbb{A}^{m_1} + \cdots + \mathbb{A}^{m_\ell}$$
is called *first-order definable* if for every $i \in \{1, \ldots, k\}$ and $j \in \{1, \ldots, \ell\}$ the set
$$\{(x, y) \in \mathbb{A}^{n_i} \times \mathbb{A}^{m_j} \mid \text{f}(x \text{ in the } i\text{-th component}) = y \text{ in the } j\text{-th component}\}$$
can be defined by a first-order formula over the vocabulary of $\mathbb{A}$.

The above is easily seen to be a category, since first-order definable functions can be composed. If the underlying structure $\mathbb{A}$ has a decidable first-order theory, then the category described above is reasonably tame, in particular one can decide if two morphisms are equal.

The single-use category admits a faithful functor to the above category. In particular, this implies that the problem of deciding if two morphisms are equal is decidable in the single-use category. □

Appendix D. Two-way automata

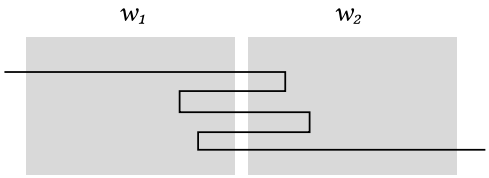
Proof of Lemma 5.1. We only need the weaker assumption, which is that the function f is finitely supported. Consider some set of atoms that supports f. This same set of atoms will also support all functions $f^{(k)}$, and it will also support their domains. In particular, the domains of these functions will form a decreasing sequence of subsets of $[\![A + X]\!]$, with all subsets having the same support. Since an orbit-finite set can have finitely many subsets with a given support, this sequence must terminate in finitely many steps. □

Proof of Theorem 5.2. The transition function in the automaton tells us how it behaves for a single input letter. We now extend this function to input strings. For every input string w, we will define a single-use function

$$\delta_w : Q + Q \to Q + Q + 1$$

that describes the behaviour of the automaton on w.

This is done by induction on the length of the input string. Suppose that we have already defined the functions for input strings w_1 and w_2. We want to compose them into a function for the string $w_1 w_2$. This function will need to account for multiple crossings of the head on the boundary between w_1 and w_2, as described in the following picture:



Taking care of these crossings is exactly what is accomplished by the trace. Indeed, consider the function

$$f : \underbrace{Q + Q}_{A} + \underbrace{Q + Q}_{X} \to \underbrace{Q + Q + 1}_{B} + \underbrace{Q + Q}_{X},$$

where A represents entering $w_1 w_2$ from either the left or right, B represents exiting $w_1 w_2$ from either the left or right or accepting, and X represents the boundary between w_1 and w_2. The trace of f is exactly the function that we need, in particular it will be undefined for inputs that lead to an infinite run.

The definition of δ_w described above describes a function

$$\text{input strings} \to \text{single-use functions of type } Q + Q \to Q + Q + 1.$$

This function is a monoid homomorphism, for a suitably defined monoid structure on the image. We use the name *transition monoid* for the output monoid. We can view the transition monoid as a linear type, namely the function space

$$Q + Q \Rightarrow Q + Q + 1.$$

Like any linear type, the transition monoid represents an orbit-finite set. This set is equipped with a monoid operation. Although this monoid operation is not single-use (because tracing is not single-use, as an operation on function spaces), it is still an equivariant operation. Therefore, we have an orbit-finite monoid. We can now use a standard fix-point algorithm on orbit-finite sets [3] to compute a representation of the sub-monoid that is generated by the images of the single letters. Finally, we can check if this sub-monoid contains an accepting element, i.e. a transition function that maps the initial configuration (initial state at the left of the word) to an accepting state. □

Appendix E. Proof of Theorem 6.4

We apply the standard proof that the category of vector spaces (without atoms and orbit-finiteness) is symmetric monoidal closed. We then observe that: (1) both evaluation and Currying are equivariant; (2) the resulting space is orbit-finitely spanned.

For the first observation (1), we simply note that the construction of the function space, as well as the evaluation and Currying morphisms, are constructed using the language of set theory, and therefore they will necessarily be equivariant [3, Equivariance Principle].

Let us now explain the second observation (2), about the function space being orbit-finitely spanned. This uses a non-trivial result from [6] which says that orbit-finitely spanned spaces are closed under duals. The space $V \Rightarrow W$ is defined to be the space of finitely supported linear maps from V to W, with the natural vector space structure. This can be seen as a subspace of a larger space, call it $V \Rrightarrow W$, which contains all linear maps, not necessarily finitely supported. A standard result in linear algebra is that $V \Rrightarrow W$ is linearly isomorphic to $V \otimes W \Rrightarrow 1$, where 1 is the field. This isomorphism is equivariant, and it preserves the property of being finitely supported. Therefore, this gives us an equivariant linear isomorphism between $V \Rightarrow W$ is isomorphic and $V \otimes W \Rightarrow 1$. The latter space is the dual of the orbit-finitely spanned space $V \otimes W$, and therefore it is orbit-finitely spanned by [6, Corollary VI.5].

We would like to remark that a similar result as Theorem 6.4 was observed in [23, Theorem 3.8], but using the smaller category of vector spaces that admit an orbit-finite basis.

References

[1] Samson Abramsky. Semantics of interaction. *arXiv preprint arXiv:1312.0121*, 2013.

[2] Quentin Aristote. Active learning of deterministic transducers with outputs in arbitrary monoids. In Aniello Murano and Alexandra Silva, editors, *32nd EACSL Annual Conference on Computer Science Logic, CSL 2024, February 19-23, 2024, Naples, Italy*, volume 288 of *LIPIcs*, pages 11:1–11:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024. doi:10.4230/LIPICS.CSL.2024.11.

[3] Mikołaj Bojańczyk. Slightly infinite sets. URL: https://www.mimuw.edu.pl/~bojan/paper/atom-book.

[4] Mikołaj Bojańczyk. Nominal Monoids. *Theory Comput. Syst.*, 53(2):194–222, 2013.

[5] Mikołaj Bojańczyk, Bartek Klin, and Slawomir Lasota. Automata theory in nominal sets. *Logical Methods in Computer Science*, 10(3), 2014.

[6] Mikołaj Bojańczyk, Bartek Klin, and Joshua Moerman. Orbit-finite-dimensional vector spaces and weighted register automata. In *36th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2021, Rome, Italy, June 29 - July 2, 2021*, pages 1–13. IEEE, 2021.

[7] Mikołaj Bojańczyk and Rafał Stefański. Single-Use Automata and Transducers for Infinite Alphabets. In Artur Czumaj, Anuj Dawar, and Emanuela Merelli, editors, *International Colloquium on Automata, Languages, and Programming (ICALP 2020)*, volume 168 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 113:1–113:14, Dagstuhl, Germany, 2020. Schloss Dagstuhl–Leibniz-Zentrum für Informatik.

[8] Pierre Clairambault. *Causal Investigations in Interactive Semantics*. Habilitation à diriger des recherches, Université Aix-Marseille, February 2024. URL: https://pageperso.lis-lab.fr/pierre.clairambault/papers/hdr.pdf.

[9] Thomas Colcombet and Daniela Petrişan. Automata minimization: a functorial approach. *Logical Methods in Computer Science*, 16, 2020.

[10] Thomas Colcombet, Daniela Petrisan, and Riccardo Stabile. Learning automata and transducers: A categorical approach. In Christel Baier and Jean Goubault-Larrecq, editors, *29th EACSL Annual Conference on Computer Science Logic, CSL 2021, January 25-28, 2021, Ljubljana, Slovenia (Virtual Conference)*, volume 183 of *LIPIcs*, pages 15:1–15:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPICS.CSL.2021.15.

[11] Valeria C. V. de Paiva. The Dialectica categories. In John W. Gray and Andre Scedrov, editors, *Categories in Computer Science and Logic*, volume 92 of *Contemporary Mathematics*, pages 47–62. American Mathematical Society, Providence, Rhode Island, 1989. Proceedings of a Summer Research Conference held June 14–20, 1987. doi:10.1090/conm/092/1003194.

[12] Paul Gallot, Aurélien Lemay, and Sylvain Salvati. Linear high-order deterministic tree transducers with regular look-ahead. In Javier Esparza and Daniel Král', editors, *45th International Symposium on Mathematical Foundations of Computer Science, MFCS 2020, August 24-28, 2020, Prague, Czech Republic*, volume 170 of *LIPIcs*, pages 38:1–38:13. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. doi:10.4230/LIPIcs.MFCS.2020.38.

[13] Jean-Yves Girard, Yves Lafont, and Laurent Regnier. *Advances in linear logic*, volume 222. Cambridge University Press, 1995.

[14] Peter Hines. A categorical framework for finite state machines. *Mathematical Structures in Computer Science*, 13(3):451–480, 2003.

[15] Martin Hyland. Game semantics. In Andrew M. Pitts and P.Editors Dybjer, editors, *Semantics and Logics of Computation*, Publications of the Newton Institute, page 131–184. Cambridge University Press, 1997. doi:10.1017/CBO9780511526619.005.

[16] Michael Kaminski and Nissim Francez. Finite-Memory Automata. *Theor. Comput. Sci.*, 134(2):329–363, 1994.

[17] Frank Neven, Thomas Schwentick, and Victor Vianu. Finite state machines for strings over infinite alphabets. *ACM Trans. Comput. Log.*, 5(3):403–435, 2004.

[18] Lê Thành Dũng Nguyễn. *Implicit automata in linear logic and categorical transducer theory*. PhD thesis, Université Paris XIII (Sorbonne Paris Nord), December 2021. URL: https://theses.hal.science/tel-04132636.

[19] Lê Thành Dũng Nguyễn, Camille Noûs, and Cécilia Pradic. Implicit automata in typed λ-calculi II: streaming transducers vs categorical semantics. *CoRR*, abs/2008.01050, 2020. arXiv:2008.01050.

[20] Lê Thành Dũng Nguyễn and Cécilia Pradic. Implicit automata in typed λ-calculi I: aperiodicity in a non-commutative logic. In Artur Czumaj, Anuj Dawar, and Emanuela Merelli, editors, *47th International Colloquium on Automata, Languages, and Programming, ICALP 2020, July 8-11, 2020, Saarbrücken, Germany (Virtual Conference)*, volume 168 of *LIPIcs*, pages 135:1–135:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. doi:10.4230/LIPIcs.ICALP.2020.135.

[21] Andrew M. Pitts. *Nominal Sets: Names and Symmetry in Computer Science*, volume 57 of *Cambridge Tracts in Theoretical Computer Science*. Cambridge University Press, 2013.

[22] Cécilia Pradic and Ian Price. Implicit automata in λ-calculi iii: affine planar string-to-string functions, 2024. arXiv:2404.03985.

[23] Michał R. Przybyłek. A note on stone-Čech compactification in zfa, 2024. arXiv:2304.09986.

[24] Alexandra Silva, Filippo Bonchi, Marcello M. Bonsangue, and Jan J. M. M. Rutten. Generalizing determinization from automata to coalgebras. *Logical Methods in Computer Science*, 9(1), 2013. doi:10.2168/LMCS-9(1:9)2013.

[25] Rafał Stefański. An automaton model for orbit-finite monoids. Master's thesis, University of Warsaw, 2018.

[26] Rafał Stefański. *The single-use restriction for register automata and transducers over infinite alphabets*. PhD thesis, University of Warsaw, 2023.